

delhivery-business-case-study

October 10, 2024

1 Business Case: Delhivery - Feature Engineering

About Delhivery

Delhivery is the largest and fastest-growing fully integrated player in India by revenue in Fiscal 2021. They aim to build the operating system for commerce, through a combination of world-class infrastructure, logistics operations of the highest quality, and cutting-edge engineering and technology capabilities.

The Data team builds intelligence and capabilities using this data that helps them to widen the gap between the quality, efficiency, and profitability of their business versus their competitors.

2 Key Goals

Clean, sanitize and manipulation of data data

Make sense out of the raw data

Feature Creation

Relationship between Features

Column Normalization /Column Standardization

Handling categorical values

Missing values - Outlier treatment / Types of outliers

```
[ ]: import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
import seaborn as sns
import statsmodels.api as sm
import math
from scipy.stats import shapiro
from scipy import stats
from scipy.stats import levene
from sklearn.preprocessing import MinMaxScaler, StandardScaler
```

```
[ ]: df = pd.read_csv('delivery.csv')
```

Basic data cleaning and exploration

1. Clean and sanitize data
2. Handle missing values in the data.
3. Analyze the structure of the data.
4. Try merging the rows using the hint mentioned above.
5. Improve understanding of data

```
[ ]: df.shape
```

```
[ ]: (144867, 24)
```

There are 144867 rows and 24 Columns in the data provided

```
[ ]: df.sample(5)
```

```
[ ]:
      data      trip_creation_time \
122154 training 2018-09-14 01:19:17.278175
94810  training 2018-09-18 06:48:21.899031
98557    test   2018-10-02 23:30:04.603660
80823  training 2018-09-15 04:22:08.308753
15945  training 2018-09-25 19:09:36.165160

      route_schedule_uuid route_type \
122154 thanos::sroute:b7971fb9-1e96-4d5d-a15d-3c55d4f...   Carting
94810  thanos::sroute:7fc0383e-7e95-456e-af48-86fea4f...   Carting
98557  thanos::sroute:bbda1b14-ddec-4bac-9585-e692bca...      FTL
80823  thanos::sroute:c12bc563-8a95-420f-87fa-41779ee...   Carting
15945  thanos::sroute:0e8d39f4-3593-430a-820e-c65b3f6...      FTL

      trip_uuid source_center \
122154 trip-153688795727790582  IND431603AAA
94810  trip-153725330189878601  IND562114AAA
98557  trip-153852300460344177  IND140001AAA
80823  trip-153698532832147425  IND401501AAA
15945  trip-153790257616491355  IND713205AAB

      source_name destination_center \
122154 Nanded_Aswningr_I (Maharashtra)  IND413531AAA
94810  Hoskote_DC (Karnataka)  IND560048AAB
98557  RoopNagar_ChotiHvl_DC (Punjab)  IND140118AAA
80823  Boisar_OstwlEmp_D (Maharashtra)  IND401602AAA
15945  Durgapur_Central_I_1 (West Bengal)  IND712311AAA

      destination_name      od_start_time ... \
122154 Latur_Srnwsngr_D (Maharashtra) 2018-09-14 01:19:17.278175 ...
```

94810	Bengaluru_Hoodi_IP (Karnataka)	2018-09-18 08:15:09.900920	...
98557	AnandprShb_PnjPiara_D (Punjab)	2018-10-03 03:57:33.549358	...
80823	Dahanu_KetyDPP_D (Maharashtra)	2018-09-15 04:22:08.308753	...
15945	Kolkata_Dankuni_HB (West Bengal)	2018-09-26 01:45:01.713299	...

	cutoff_timestamp	actual_distance_to_destination	actual_time \
122154	2018-09-14 04:29:55	64.094012	114.0
94810	2018-09-18 08:37:20	9.363827	30.0
98557	2018-10-03 04:44:15	23.086353	39.0
80823	2018-09-15 05:11:56	20.318517	52.0
15945	2018-09-26 04:59:57	66.058826	84.0

	osrm_time	osrm_distance	factor	segment_actual_time \
122154	55.0	77.9102	2.072727	16.0
94810	17.0	19.8133	1.764706	30.0
98557	20.0	26.8962	1.950000	39.0
80823	25.0	27.6221	2.080000	3.0
15945	50.0	70.9373	1.680000	29.0

	segment_osrm_time	segment_osrm_distance	segment_factor
122154	7.0	10.4938	2.285714
94810	17.0	19.8133	1.764706
98557	20.0	26.8962	1.950000
80823	1.0	1.6881	3.000000
15945	17.0	23.4562	1.705882

[5 rows x 24 columns]

```
[ ]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144867 entries, 0 to 144866
Data columns (total 24 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   data                                  144867 non-null object
1   trip_creation_time                    144867 non-null object
2   route_schedule_uuid                   144867 non-null object
3   route_type                            144867 non-null object
4   trip_uuid                             144867 non-null object
5   source_center                         144867 non-null object
6   source_name                           144574 non-null object
7   destination_center                    144867 non-null object
8   destination_name                       144606 non-null object
9   od_start_time                         144867 non-null object
10  od_end_time                           144867 non-null object
11  start_scan_to_end_scan                 144867 non-null float64
```

12	is_cutoff	144867	non-null	bool
13	cutoff_factor	144867	non-null	int64
14	cutoff_timestamp	144867	non-null	object
15	actual_distance_to_destination	144867	non-null	float64
16	actual_time	144867	non-null	float64
17	osrm_time	144867	non-null	float64
18	osrm_distance	144867	non-null	float64
19	factor	144867	non-null	float64
20	segment_actual_time	144867	non-null	float64
21	segment_osrm_time	144867	non-null	float64
22	segment_osrm_distance	144867	non-null	float64
23	segment_factor	144867	non-null	float64

dtypes: bool(1), float64(10), int64(1), object(12)

memory usage: 25.6+ MB

Here is the key findings:

- I) boolean (1): is_cutoff
- II) int64 (1): cutoff_factor
- III) float64 (10):
 - 1) start_scan_to_end_scan
 - 2) actual_distance_to_destination
 - 3) actual_time
 - 4) osrm_time
 - 5) osrm_distance
 - 6) factor
 - 7) segment_actual_time
 - 8) segment_osrm_time
 - 9) segment_osrm_distance
 - 10) segment_factor
- III) object (12):
 - 1) data
 - 2) trip_creation_time
 - 3) route_schedule_uuid
 - 4) route_type
 - 5) trip_uuid
 - 6) source_center
 - 7) source_name
 - 8) destination_center
 - 9) destination_name
 - 10) od_start_time
 - 11) od_end_time
 - 12) cutoff_timestamp

Finding duplicate values

```
[ ]: df.duplicated().sum()
```

```
[ ]: 0
```

Getting missing value count

```
[ ]: print(df.isnull().sum())
```

data	0
trip_creation_time	0
route_schedule_uuid	0
route_type	0
trip_uuid	0
source_center	0
source_name	293
destination_center	0
destination_name	261
od_start_time	0
od_end_time	0
start_scan_to_end_scan	0
is_cutoff	0
cutoff_factor	0
cutoff_timestamp	0
actual_distance_to_destination	0
actual_time	0
osrm_time	0
osrm_distance	0
factor	0
segment_actual_time	0
segment_osrm_time	0
segment_osrm_distance	0
segment_factor	0
dtype: int64	

Checking unique values

```
[ ]: df.nunique()
```

data	2
trip_creation_time	14817
route_schedule_uuid	1504
route_type	2
trip_uuid	14817
source_center	1508
source_name	1498
destination_center	1481
destination_name	1468
od_start_time	26369

od_end_time	26369
start_scan_to_end_scan	1915
is_cutoff	2
cutoff_factor	501
cutoff_timestamp	93180
actual_distance_to_destination	144515
actual_time	3182
osrm_time	1531
osrm_distance	138046
factor	45641
segment_actual_time	747
segment_osrm_time	214
segment_osrm_distance	113799
segment_factor	5675
dtype: int64	

The given data has two kinds of data that Training and test data

```
[ ]: print("No of rows in training data : ",df[df["data"]=="training"].data.count())

print("No of rows in test data : ",df[df["data"]=="test"].data.count())
```

No of rows in training data : 104858

No of rows in test data : 40009

Key Observations:

Data Shape: The DataFrame contains 144,867 rows and 24 columns.

Columns and Data Types: The dataset includes a variety of data types, such as strings (object), floats (float64), booleans (bool), and integers (int64).

Missing Values: Two columns have missing values: source_name has 293 missing entries, while destination_name has 261 missing entries.

Duplicate Values: There are no duplicate records in the dataset.

Unique Values: Most columns exhibit a high number of unique values, indicating a diverse dataset. For instance, trip_creation_time has 14,817 unique entries, source_center contains 1,508 unique values, and actual_distance_to_destination has 144,515 unique measurements.

Time Columns: The columns od_start_time and od_end_time each feature approximately 26,369 unique values, suggesting a high level of granularity or a wide time range.

Rows Breakdown: There are 104,858 rows in the training data and 40,009 rows in the test data, indicating that the majority of the dataset is comprised of training data.

Dropping unknown fields

```
[ ]: df.columns
```

```
[ ]: Index(['data', 'trip_creation_time', 'route_schedule_uuid', 'route_type',
          'trip_uuid', 'source_center', 'source_name', 'destination_center',
          'destination_name', 'od_start_time', 'od_end_time',
          'start_scan_to_end_scan', 'is_cutoff', 'cutoff_factor',
          'cutoff_timestamp', 'actual_distance_to_destination', 'actual_time',
          'osrm_time', 'osrm_distance', 'factor', 'segment_actual_time',
          'segment_osrm_time', 'segment_osrm_distance', 'segment_factor'],
          dtype='object')
```

Here are the fields we are removing from the dataset as they are not clearly defined or do not contribute meaningfully to our calculations:

is_cutoff, cutoff_factor, cutoff_timestamp, factor, and segment_factor.

```
[ ]: unique_columns = ['is_cutoff', 'cutoff_factor', 'cutoff_timestamp', 'factor',
                       'segment_factor']
df = df.drop(columns=unique_columns)
```

```
[ ]: df.dtypes
```

```
[ ]: data
trip_creation_time      object
route_schedule_uuid     object
route_type              object
trip_uuid              object
source_center           object
source_name             object
destination_center      object
destination_name        object
od_start_time           object
od_end_time            object
start_scan_to_end_scan  float64
actual_distance_to_destination float64
actual_time             float64
osrm_time               float64
osrm_distance           float64
segment_actual_time     float64
segment_osrm_time       float64
segment_osrm_distance   float64
dtype: object
```

Converting data type of features:

- 1) We can convert data, route_type to categories
- 2) Also we can convert all features containing time data to time type

```
[ ]: category_col = ['data', 'route_type']
for i in category_col:
    print(f'Unique values in {i} column:', df[i].unique())
```

```
del category_col
```

Unique values in data column: ['training' 'test']

Unique values in route_type column: ['Carting' 'FTL']

```
[ ]: # df['data'] = df['data'].astype('category')
# df['route_type'] = df['route_type'].astype('category')

category_col = ['data', 'route_type']
for i in category_col:
    df[i] = df[i].astype('category')

del category_col
```

```
[ ]: df.info()
```

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 144867 entries, 0 to 144866

Data columns (total 19 columns):

#	Column	Non-Null Count	Dtype
0	data	144867 non-null	category
1	trip_creation_time	144867 non-null	object
2	route_schedule_uuid	144867 non-null	object
3	route_type	144867 non-null	category
4	trip_uuid	144867 non-null	object
5	source_center	144867 non-null	object
6	source_name	144574 non-null	object
7	destination_center	144867 non-null	object
8	destination_name	144606 non-null	object
9	od_start_time	144867 non-null	object
10	od_end_time	144867 non-null	object
11	start_scan_to_end_scan	144867 non-null	float64
12	actual_distance_to_destination	144867 non-null	float64
13	actual_time	144867 non-null	float64
14	osrm_time	144867 non-null	float64
15	osrm_distance	144867 non-null	float64
16	segment_actual_time	144867 non-null	float64
17	segment_osrm_time	144867 non-null	float64
18	segment_osrm_distance	144867 non-null	float64

dtypes: category(2), float64(8), object(9)

memory usage: 19.1+ MB

Converting the time columns

```
[ ]: df[['trip_creation_time', 'od_start_time', 'od_end_time', 'actual_time',
        'osrm_time', 'segment_actual_time', 'segment_osrm_time']]
```



```
[ ]:      trip_creation_time      od_start_time \
0      2018-09-20 02:35:36.476840 2018-09-20 03:21:32.418600
1      2018-09-20 02:35:36.476840 2018-09-20 03:21:32.418600
2      2018-09-20 02:35:36.476840 2018-09-20 03:21:32.418600
3      2018-09-20 02:35:36.476840 2018-09-20 03:21:32.418600
4      2018-09-20 02:35:36.476840 2018-09-20 03:21:32.418600
...
144862 2018-09-20 16:24:28.436231 2018-09-20 16:24:28.436231
144863 2018-09-20 16:24:28.436231 2018-09-20 16:24:28.436231
144864 2018-09-20 16:24:28.436231 2018-09-20 16:24:28.436231
144865 2018-09-20 16:24:28.436231 2018-09-20 16:24:28.436231
144866 2018-09-20 16:24:28.436231 2018-09-20 16:24:28.436231

      od_end_time  actual_time  osrm_time \
0      2018-09-20 04:47:45.236797      14.0      11.0
1      2018-09-20 04:47:45.236797      24.0      20.0
2      2018-09-20 04:47:45.236797      40.0      28.0
3      2018-09-20 04:47:45.236797      62.0      40.0
4      2018-09-20 04:47:45.236797      68.0      44.0
...
144862 2018-09-20 23:32:09.618069      94.0      60.0
144863 2018-09-20 23:32:09.618069     120.0      76.0
144864 2018-09-20 23:32:09.618069     140.0      88.0
144865 2018-09-20 23:32:09.618069     158.0      98.0
144866 2018-09-20 23:32:09.618069     426.0      95.0

      segment_actual_time  segment_osrm_time
0              14.0              11.0
1              10.0              9.0
2              16.0              7.0
3              21.0             12.0
4               6.0              5.0
...
144862              12.0             12.0
144863              26.0             21.0
144864              20.0             34.0
144865              17.0             27.0
144866             268.0              9.0
```

[144867 rows x 7 columns]

```
[ ]: time_columns = ['trip_creation_time', 'od_start_time', 'od_end_time']
for i in time_columns:
    df[i] = pd.to_datetime(df[i])
del time_columns
```

```
[ ]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144867 entries, 0 to 144866
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   data                                  144867 non-null  category
1   trip_creation_time                   144867 non-null  datetime64[ns]
2   route_schedule_uuid                 144867 non-null  object
3   route_type                           144867 non-null  category
4   trip_uuid                            144867 non-null  object
5   source_center                       144867 non-null  object
6   source_name                         144574 non-null  object
7   destination_center                  144867 non-null  object
8   destination_name                    144606 non-null  object
9   od_start_time                       144867 non-null  datetime64[ns]
10  od_end_time                         144867 non-null  datetime64[ns]
11  start_scan_to_end_scan               144867 non-null  float64
12  actual_distance_to_destination       144867 non-null  float64
13  actual_time                         144867 non-null  float64
14  osrm_time                           144867 non-null  float64
15  osrm_distance                       144867 non-null  float64
16  segment_actual_time                 144867 non-null  float64
17  segment_osrm_time                   144867 non-null  float64
18  segment_osrm_distance               144867 non-null  float64
dtypes: category(2), datetime64[ns](3), float64(8), object(6)
memory usage: 19.1+ MB
```

Time period (Min and Max) of the given dataset

```
[ ]: print('Trip Creation Time: ', df['trip_creation_time'].min(), 'to',
      ↪df['trip_creation_time'].max())
print('Od Start Time: ', df['od_start_time'].min(), 'to', df['od_start_time'].
      ↪max())
print('Od End Time: ', df['od_end_time'].min(), 'to', df['od_end_time'].max())

print('====' * 50)

print('Start time: ', df['trip_creation_time'].min())
print('End time: ', df['od_end_time'].max())
print('Total time period: ', df['od_end_time'].max() - df['trip_creation_time'].
      ↪min())
```

Trip Creation Time: 2018-09-12 00:00:16.535741 to 2018-10-03 23:59:42.701692

Od Start Time: 2018-09-12 00:00:16.535741 to 2018-10-06 04:27:23.392375

Od End Time: 2018-09-12 00:50:10.814399 to 2018-10-08 03:00:24.353479

```
=====
=====
=====
```

Start time: 2018-09-12 00:00:16.535741
 End time: 2018-10-08 03:00:24.353479
 Total time period: 26 days 03:00:07.817738

Handling Missing Values

```
[ ]: df.isnull().sum()
```

```
[ ]: data
trip_creation_time      0
route_schedule_uuid    0
route_type              0
trip_uuid              0
source_center           0
source_name             293
destination_center      0
destination_name        261
od_start_time           0
od_end_time             0
start_scan_to_end_scan  0
actual_distance_to_destination 0
actual_time             0
osrm_time               0
osrm_distance           0
segment_actual_time     0
segment_osrm_time       0
segment_osrm_distance   0
dtype: int64
```

As we can observe source_name has 293 missing data points and destination_name has 261 missing data points

```
[ ]: df[['source_name', 'source_center', 'destination_name', 'destination_center']].
      ↪sample(10)
```

```
[ ]:
source_name source_center \
57731      Surat_HUB (Gujarat) IND395023AAA
106358      Kanchipuram_DC (Tamil Nadu) IND631501AAA
52561  PaontSahib_Gurudwar_D (Himachal Pradesh) IND173025AAA
41999      Mangalore_Kuntikna_H (Karnataka) IND575004AAB
37408      Hyderabad_Shamshbd_H (Telangana) IND501359AAE
83254      Gola_BkgnRoad_D (Uttar Pradesh) IND262802AAA
63460      Khedbrahma_Patelfli_D (Gujarat) IND383255AAB
78498      Barmer_Nehru3PL_D (Rajasthan) IND344001AAA
131801      Bengaluru_Hoodi_IP (Karnataka) IND560048AAB
71687      Noida_Sec-83_DC (Uttar Pradesh) IND201304AAA

destination_name destination_center
```

57731	Vapi_IndEstat_I (Gujarat)	IND396191AAC
106358	MAA_Poonamallee_HB (Tamil Nadu)	IND600056AAB
52561	Chandigarh_Mehmdpur_H (Punjab)	IND160002AAC
41999	Bangalore_Nelmngla_H (Karnataka)	IND562132AAA
37408	Solapur_Central_I_2 (Maharashtra)	IND413002AAA
83254	Dhaurahara_TahsilRD_D (Uttar Pradesh)	IND262723AAB
63460	Mehsana_Panchot_IP (Gujarat)	IND384205AAA
78498	Phalodi_PalikDPP_D (Rajasthan)	IND342301AAA
131801	Bengaluru_Bomsndra_HB (Karnataka)	IND560099AAB
71687	Noida_Sec 02_DPC (Uttar Pradesh)	IND201301AAF

```
[ ]: df['source_name'].fillna(df['source_name'].mode()[0], inplace=True)
df['destination_name'].fillna(df['destination_name'].mode()[0], inplace=True)
```

<ipython-input-25-3518f414579a>:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['source_name'].fillna(df['source_name'].mode()[0], inplace=True)
<ipython-input-25-3518f414579a>:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
```

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['destination_name'].fillna(df['destination_name'].mode()[0], inplace=True)
```

```
[ ]: df.isnull().sum()
```

```
[ ]: data
trip_creation_time    0
route_schedule_uuid   0
route_type            0
```

```

trip_uid      0
source_center 0
source_name    0
destination_center 0
destination_name 0
od_start_time 0
od_end_time    0
start_scan_to_end_scan 0
actual_distance_to_destination 0
actual_time    0
osrm_time      0
osrm_distance  0
segment_actual_time 0
segment_osrm_time 0
segment_osrm_distance 0
dtype: int64

```

Visual Analysis (Distribution of plots of all continuous variables, box plot of all categorical variables)

```

[ ]: # Distribution Plots for Continuous Variables

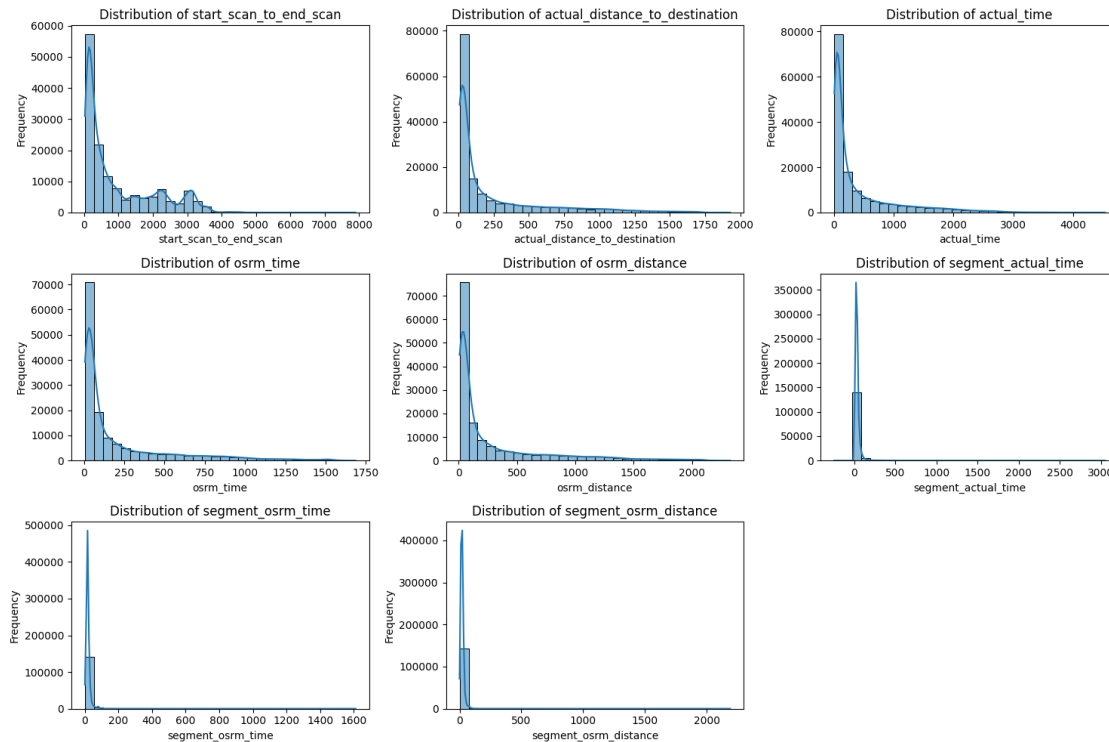
plt.figure(figsize=(15, 10))

# List of continuous variables
continuous_vars = ['start_scan_to_end_scan', 'actual_distance_to_destination',
                  'actual_time', 'osrm_time', 'osrm_distance',
                  'segment_actual_time', 'segment_osrm_time',
                  'segment_osrm_distance']

# Loop through continuous variables to create distribution plots
for i, var in enumerate(continuous_vars):
    plt.subplot(3, 3, i + 1) # Adjust based on number of variables
    sns.histplot(df[var], kde=True, bins=30) # Use sns.kdeplot for KDE only
    plt.title(f'Distribution of {var}')
    plt.xlabel(var)
    plt.ylabel('Frequency')

plt.tight_layout()
plt.show()

```



```
[ ]: # Set up the figure size
plt.figure(figsize=(10, 6))

# Create the bar plot with a color palette
sns.countplot(data=df, x='route_type', palette='Set2', order=df['route_type'].
    ↪value_counts().index)

# Set plot title and labels
plt.title('Count of Each Route Type')
plt.xlabel('Route Type')
plt.ylabel('Count')

# Show the plot
plt.xticks(rotation=45) # Rotate x-axis labels for better visibility
plt.show()
```

<ipython-input-28-d8bec46647f6>:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

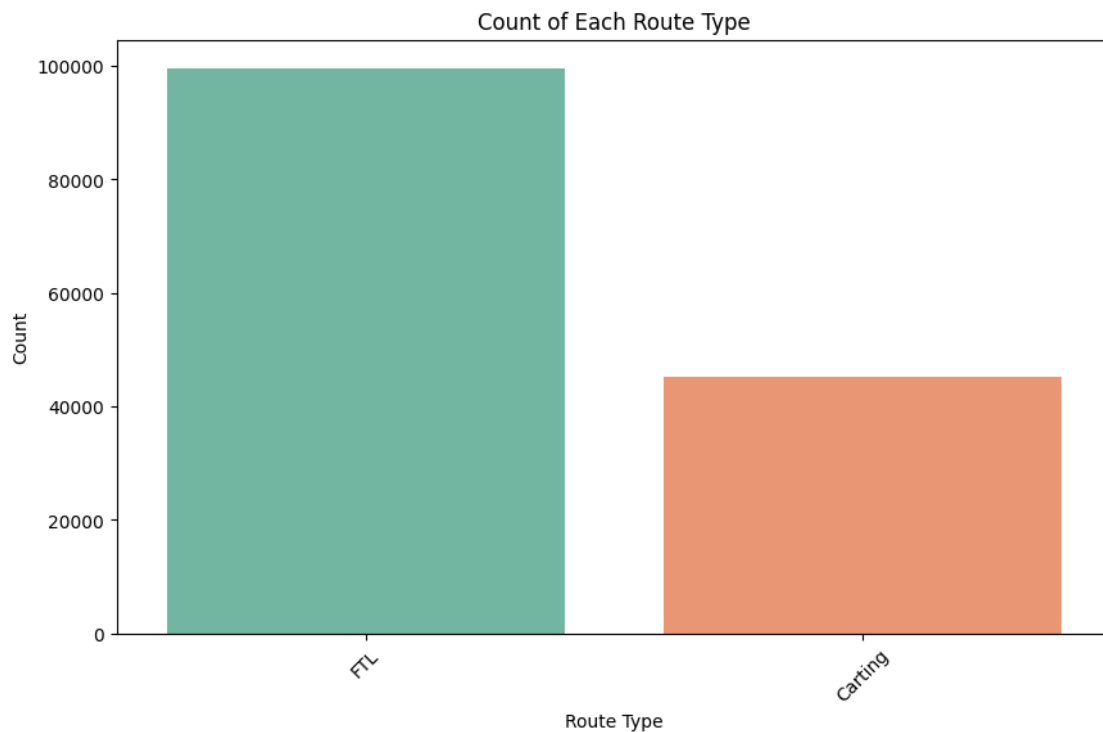
```
sns.countplot(data=df, x='route_type', palette='Set2',
order=df['route_type'].value_counts().index)
```

/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning:
When grouping with a length-1 list-like, you will need to pass a length-1 tuple
to get_group in a future version of pandas. Pass `(name,)` instead of `name` to
silence this warning.

```
data_subset = grouped_data.get_group(pd_key)
```

/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning:
When grouping with a length-1 list-like, you will need to pass a length-1 tuple
to get_group in a future version of pandas. Pass `(name,)` instead of `name` to
silence this warning.

```
data_subset = grouped_data.get_group(pd_key)
```



```
[ ]: # Set up the figure size
plt.figure(figsize=(10, 6))

# Create the bar plot with a color palette
sns.countplot(data=df, x='data', palette='Set2', order=df['data'].
    ↳value_counts().index)

# Set plot title and labels
plt.title('Count of Each data')
plt.xlabel('Data')
plt.ylabel('Count')

# Show the plot
```

```
plt.xticks(rotation=45) # Rotate x-axis labels for better visibility
plt.show()
```

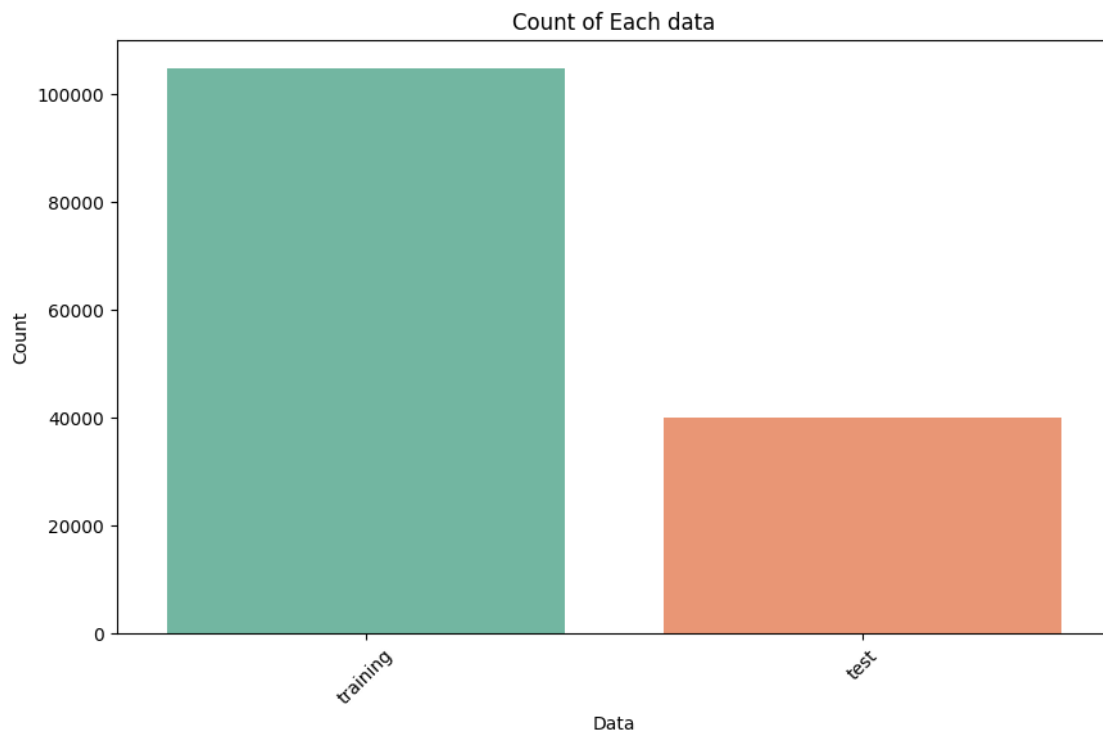
<ipython-input-29-5e9ecbcc7188>:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(data=df, x='data', palette='Set2',
order=df['data'].value_counts().index)
/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning:
When grouping with a length-1 list-like, you will need to pass a length-1 tuple
to get_group in a future version of pandas. Pass `(name,)` instead of `name` to
silence this warning.
```

```
data_subset = grouped_data.get_group(pd_key)
/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning:
When grouping with a length-1 list-like, you will need to pass a length-1 tuple
to get_group in a future version of pandas. Pass `(name,)` instead of `name` to
silence this warning.
```

```
data_subset = grouped_data.get_group(pd_key)
```

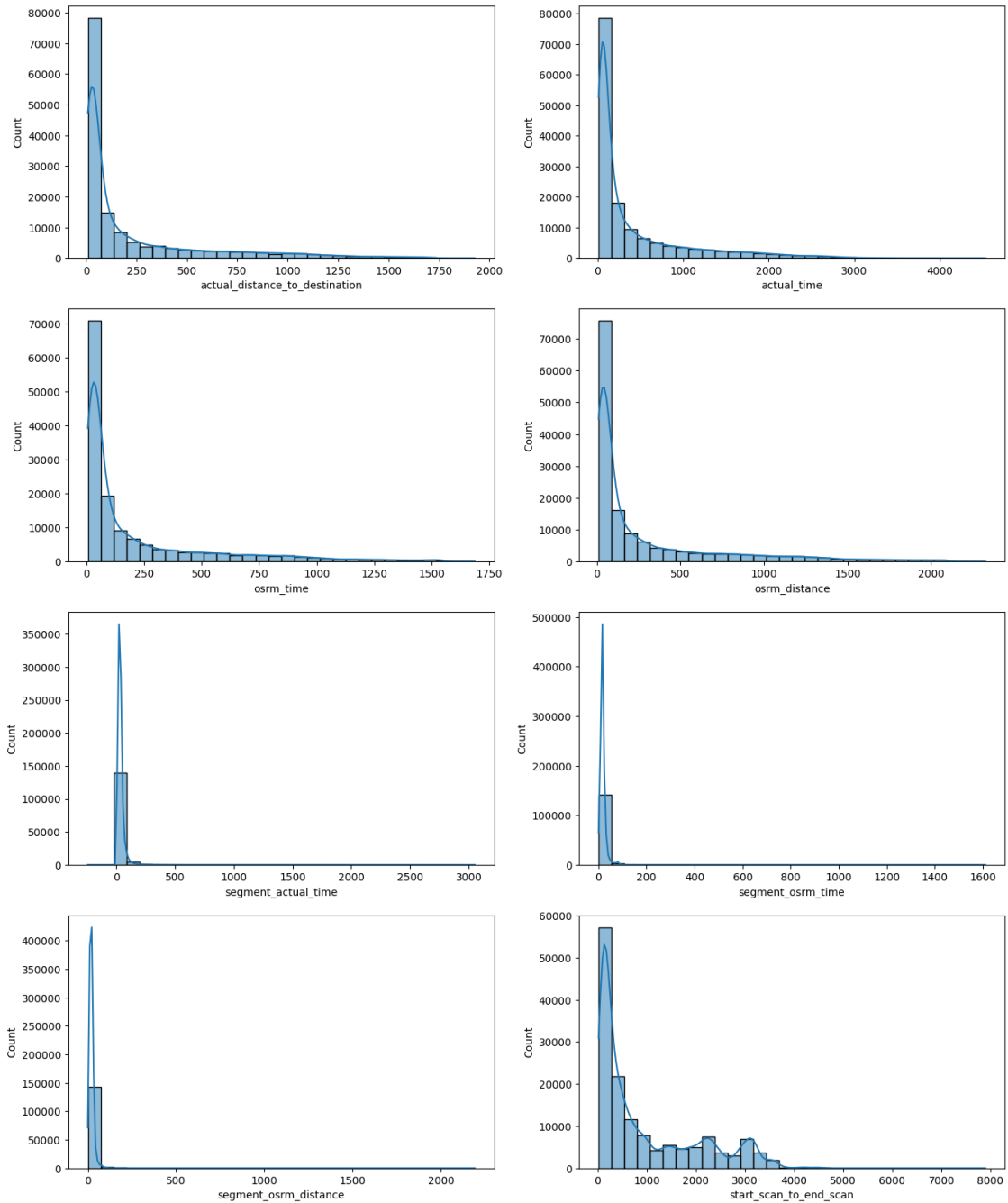


Distribution of Numerical Variables


```
[ ]: fix, ax = plt.subplots(nrows = 4, ncols = 2, figsize=(16, 20))

sns.histplot(data=df, x='actual_distance_to_destination', ax=ax[0][0],
             ↪kde=True, bins=30)
sns.histplot(data=df, x='actual_time', ax=ax[0][1], kde=True, bins=30)
sns.histplot(data=df, x='osrm_time', ax=ax[1][0], kde=True, bins=30)
sns.histplot(data=df, x='osrm_distance', ax=ax[1][1], kde=True, bins=30)
sns.histplot(data=df, x='segment_actual_time', ax=ax[2][0], kde=True, bins=30)
sns.histplot(data=df, x='segment_osrm_time', ax=ax[2][1], kde=True, bins=30)
sns.histplot(data=df, x='segment_osrm_distance', ax=ax[3][0], kde=True, bins=30)
sns.histplot(data=df, x='start_scan_to_end_scan', ax=ax[3][1], kde=True,
             ↪bins=30)

plt.show()
```



```
[ ]: fix, ax = plt.subplots(nrows = 4, ncols = 2, figsize=(16, 20))

sns.violinplot(data=df, x='actual_distance_to_destination', ax=ax[0][0])
sns.violinplot(data=df, x='actual_time', ax=ax[0][1])
sns.violinplot(data=df, x='osrm_time', ax=ax[1][0])
sns.violinplot(data=df, x='osrm_distance', ax=ax[1][1])
sns.violinplot(data=df, x='segment_actual_time', ax=ax[2][0])
```

```
sns.violinplot(data=df, x='segment_osrm_time', ax=ax[2][1])
sns.violinplot(data=df, x='segment_osrm_distance', ax=ax[3][0])
sns.violinplot(data=df, x='start_scan_to_end_scan', ax=ax[3][1])

plt.show()
```

/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning:
When grouping with a length-1 list-like, you will need to pass a length-1 tuple
to get_group in a future version of pandas. Pass `(name,)` instead of `name` to
silence this warning.

```
data_subset = grouped_data.get_group(pd_key)
```

/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning:
When grouping with a length-1 list-like, you will need to pass a length-1 tuple
to get_group in a future version of pandas. Pass `(name,)` instead of `name` to
silence this warning.

```
data_subset = grouped_data.get_group(pd_key)
```

/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning:
When grouping with a length-1 list-like, you will need to pass a length-1 tuple
to get_group in a future version of pandas. Pass `(name,)` instead of `name` to
silence this warning.

```
data_subset = grouped_data.get_group(pd_key)
```

/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning:
When grouping with a length-1 list-like, you will need to pass a length-1 tuple
to get_group in a future version of pandas. Pass `(name,)` instead of `name` to
silence this warning.

```
data_subset = grouped_data.get_group(pd_key)
```

/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning:
When grouping with a length-1 list-like, you will need to pass a length-1 tuple
to get_group in a future version of pandas. Pass `(name,)` instead of `name` to
silence this warning.

```
data_subset = grouped_data.get_group(pd_key)
```

/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning:
When grouping with a length-1 list-like, you will need to pass a length-1 tuple
to get_group in a future version of pandas. Pass `(name,)` instead of `name` to
silence this warning.

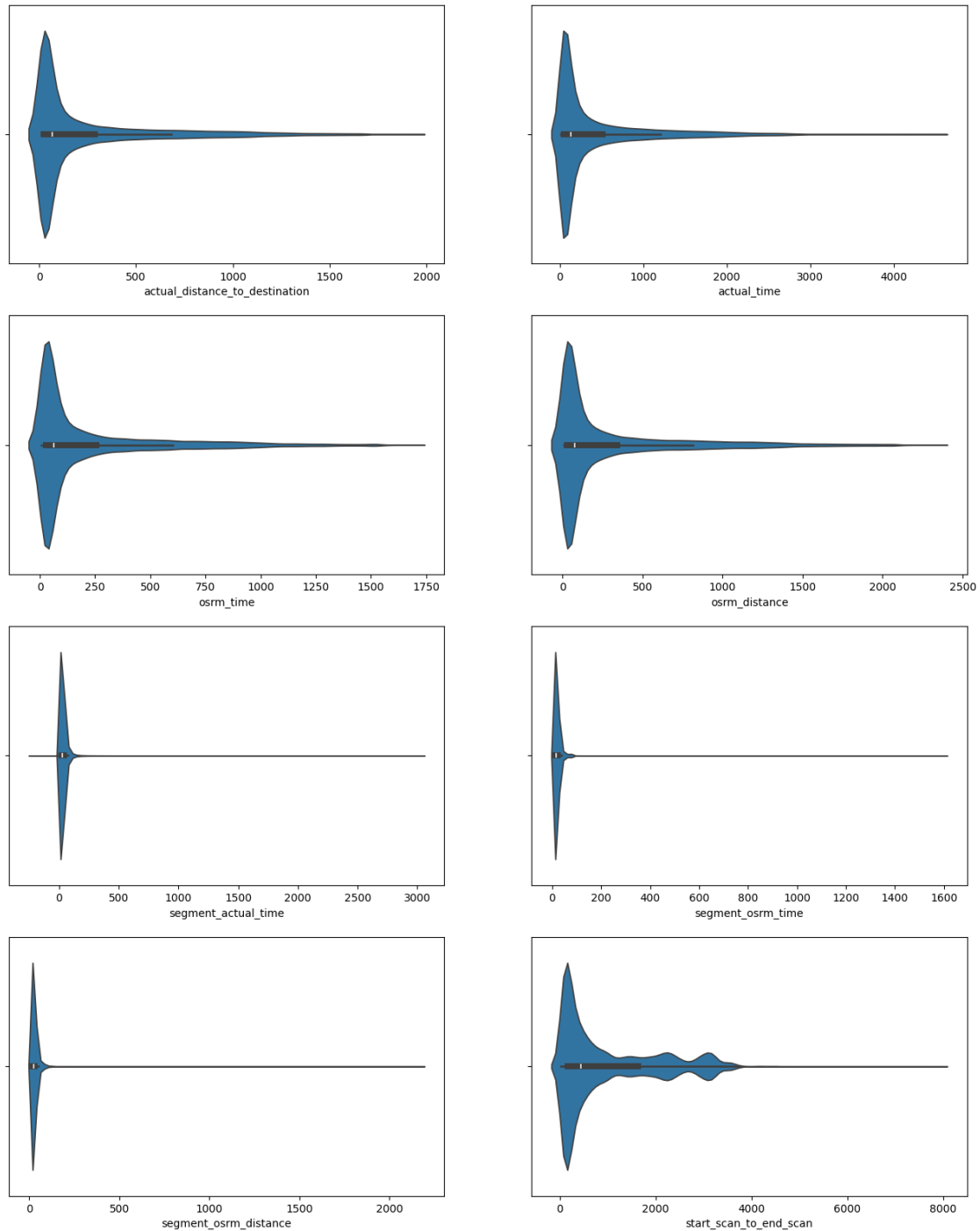
```
data_subset = grouped_data.get_group(pd_key)
```

/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning:
When grouping with a length-1 list-like, you will need to pass a length-1 tuple
to get_group in a future version of pandas. Pass `(name,)` instead of `name` to
silence this warning.

```
data_subset = grouped_data.get_group(pd_key)
```

/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning:
When grouping with a length-1 list-like, you will need to pass a length-1 tuple
to get_group in a future version of pandas. Pass `(name,)` instead of `name` to
silence this warning.

```
data_subset = grouped_data.get_group(pd_key)
```



Visually inspecting the data from above graph shows that most of the data is right skewed

```
[ ]: #   Box Plots for Categorical Variables

plt.figure(figsize=(15, 10))
```

```

# List of categorical variables
categorical_vars = ['route_type', 'source_name', 'destination_name'] # Add
↳ more as needed

# Loop through categorical variables to create box plots
for i, cat_var in enumerate(categorical_vars):
    plt.subplot(2, 2, i + 1) # Adjust based on number of categorical variables
    sns.boxplot(data=df, x=cat_var, y='actual_time') # Change 'actual_time' to
↳ the continuous variable of interest
    plt.title(f'Box Plot of {cat_var} vs Actual Time')
    plt.xlabel(cat_var)
    plt.ylabel('Actual Time')

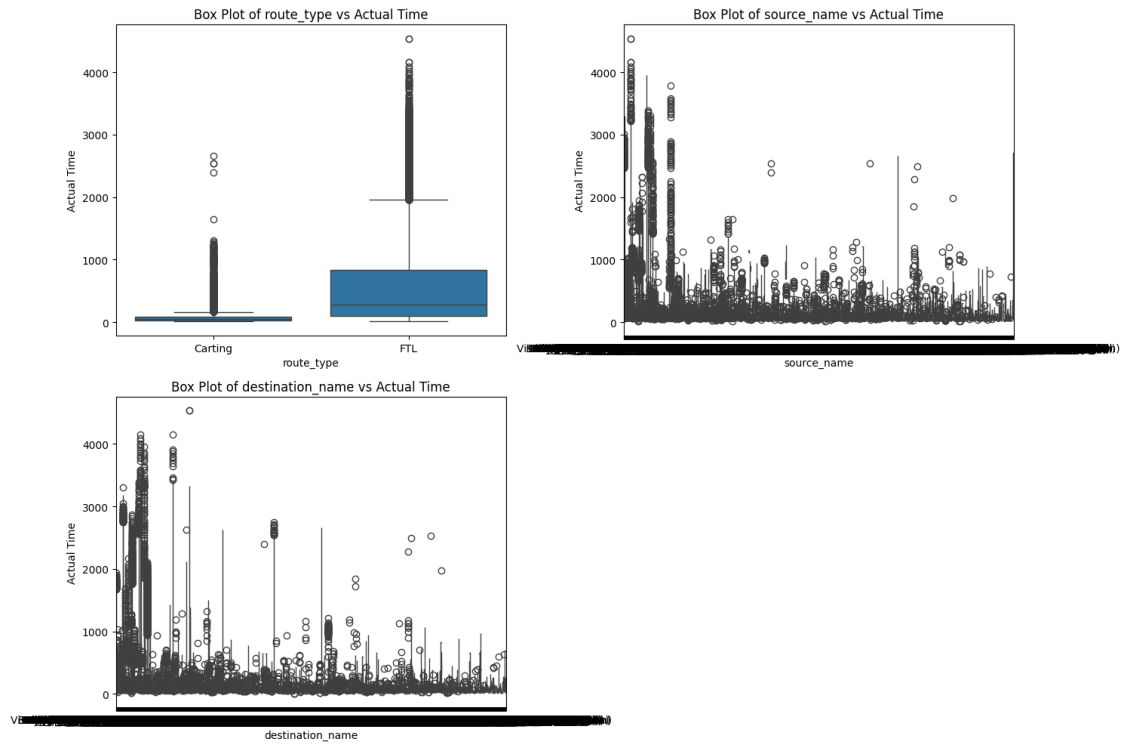
plt.tight_layout()
plt.show()

```

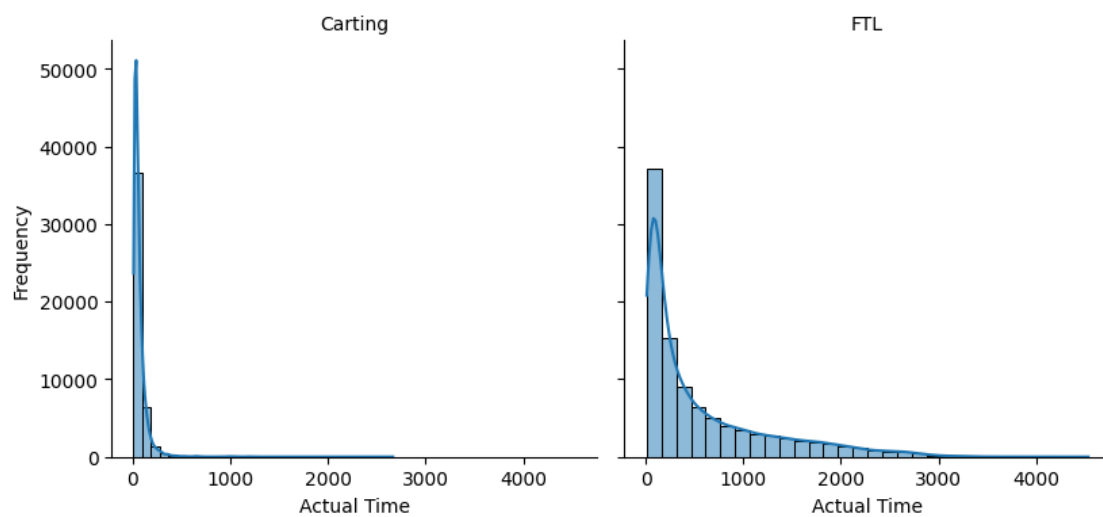
```

/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:
FutureWarning: SeriesGroupBy.grouper is deprecated and will be removed in a
future version of pandas.
    positions = grouped.grouper.result_index.to_numpy(dtype=float)
/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:
FutureWarning: SeriesGroupBy.grouper is deprecated and will be removed in a
future version of pandas.
    positions = grouped.grouper.result_index.to_numpy(dtype=float)
/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:
FutureWarning: SeriesGroupBy.grouper is deprecated and will be removed in a
future version of pandas.
    positions = grouped.grouper.result_index.to_numpy(dtype=float)

```



```
[ ]: # Set up a FacetGrid for distribution plots based on 'route_type'
g = sns.FacetGrid(df, col='route_type', col_wrap=4, height=4)
g.map(sns.histplot, 'actual_time', kde=True, bins=30)
g.set_axis_labels('Actual Time', 'Frequency')
g.set_titles(col_template="{col_name}")
plt.show()
```



```
[ ]: # Set up a FacetGrid for box plots based on 'source_name'
# Count the occurrences of each source_name and filter for the top N categories
top_n = 10 # Adjust this number as needed
top_source_names = df['source_name'].value_counts().nlargest(top_n).index

# Filter the DataFrame for only the top source names
filtered_df = df[df['source_name'].isin(top_source_names)]

# Set up a FacetGrid for box plots based on the filtered source_name
plt.figure(figsize=(15, 10))
g = sns.FacetGrid(filtered_df, col='source_name', col_wrap=4, height=4)
g.map(sns.boxplot, 'route_type', 'actual_time') # Change 'actual_time' if
needed
g.set_axis_labels('Route Type', 'Actual Time')
g.set_titles(col_template="{col_name}")
plt.show()
```

/usr/local/lib/python3.10/dist-packages/seaborn/axisgrid.py:718: UserWarning:
Using the boxplot function without specifying `order` is likely to produce an
incorrect plot.

warnings.warn(warning)

/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:

FutureWarning: SeriesGroupBy.grouper is deprecated and will be removed in a
future version of pandas.

positions = grouped.grouper.result_index.to_numpy(dtype=float)

/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:

FutureWarning: SeriesGroupBy.grouper is deprecated and will be removed in a
future version of pandas.

positions = grouped.grouper.result_index.to_numpy(dtype=float)

/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:

FutureWarning: SeriesGroupBy.grouper is deprecated and will be removed in a
future version of pandas.

positions = grouped.grouper.result_index.to_numpy(dtype=float)

/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:

FutureWarning: SeriesGroupBy.grouper is deprecated and will be removed in a
future version of pandas.

positions = grouped.grouper.result_index.to_numpy(dtype=float)

/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:

FutureWarning: SeriesGroupBy.grouper is deprecated and will be removed in a
future version of pandas.

positions = grouped.grouper.result_index.to_numpy(dtype=float)

/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:

FutureWarning: SeriesGroupBy.grouper is deprecated and will be removed in a
future version of pandas.

positions = grouped.grouper.result_index.to_numpy(dtype=float)

/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:

FutureWarning: SeriesGroupBy.grouper is deprecated and will be removed in a

future version of pandas.

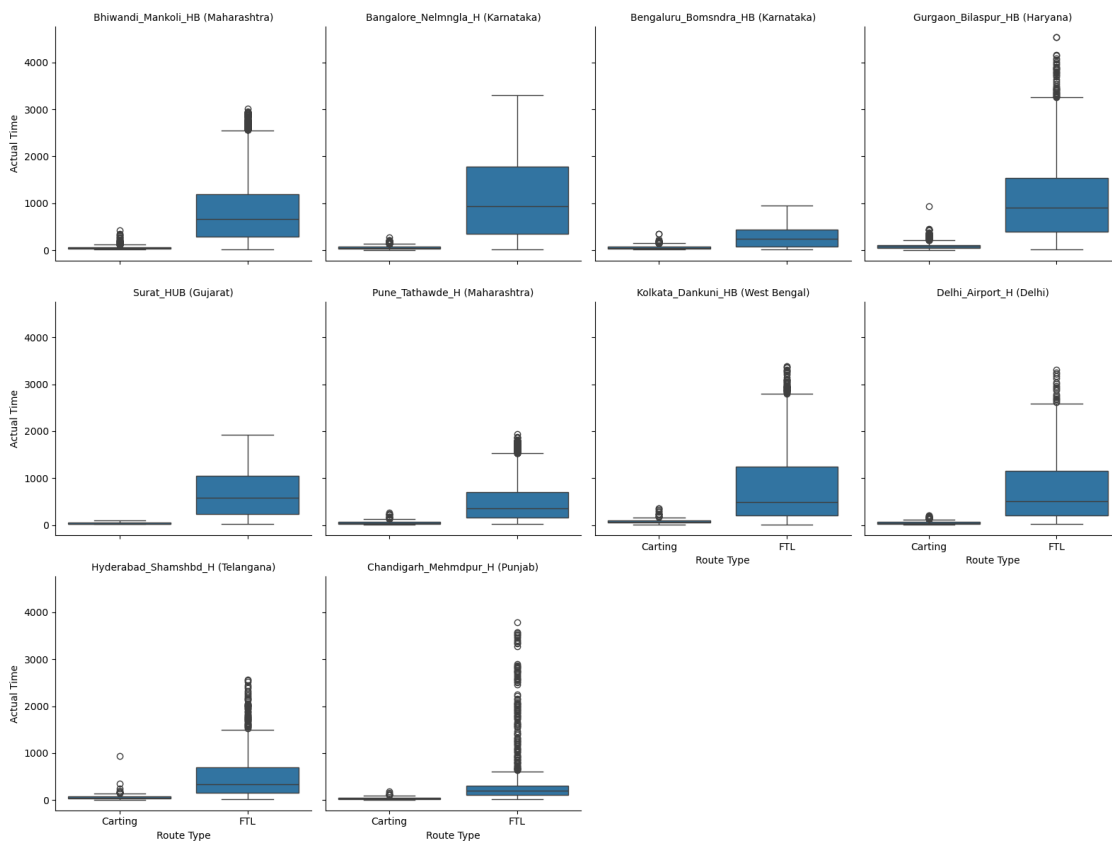
```
positions = grouped.grouper.result_index.to_numpy(dtype=float)
/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:
FutureWarning: SeriesGroupBy.grouper is deprecated and will be removed in a
future version of pandas.
```

```
positions = grouped.grouper.result_index.to_numpy(dtype=float)
/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:
FutureWarning: SeriesGroupBy.grouper is deprecated and will be removed in a
future version of pandas.
```

```
positions = grouped.grouper.result_index.to_numpy(dtype=float)
/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:
FutureWarning: SeriesGroupBy.grouper is deprecated and will be removed in a
future version of pandas.
```

```
positions = grouped.grouper.result_index.to_numpy(dtype=float)
```

<Figure size 1500x1000 with 0 Axes>



```
[ ]: df.columns
```

```
[ ]: Index(['data', 'trip_creation_time', 'route_schedule_uuid', 'route_type',
          'trip_uuid', 'source_center', 'source_name', 'destination_center',
```



```

'destination_name', 'od_start_time', 'od_end_time',
'start_scan_to_end_scan', 'actual_distance_to_destination',
'actual_time', 'osrm_time', 'osrm_distance', 'segment_actual_time',
'segment_osrm_time', 'segment_osrm_distance'],
dtype='object')

```

```

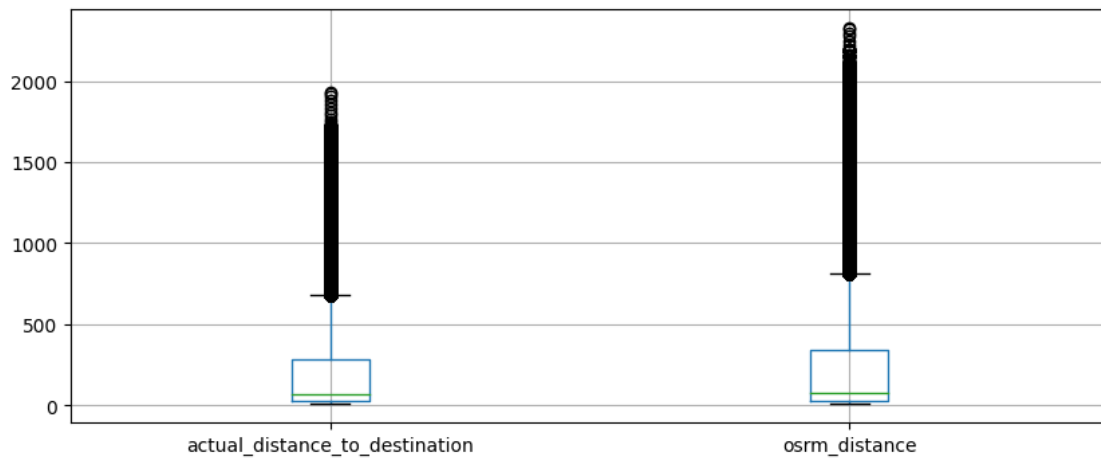
[ ]: # Boxplots acutal distance and start scan to end scan
df[['actual_distance_to_destination', 'osrm_distance']].boxplot(figsize=(10,4))

```

```

[ ]: <Axes: >

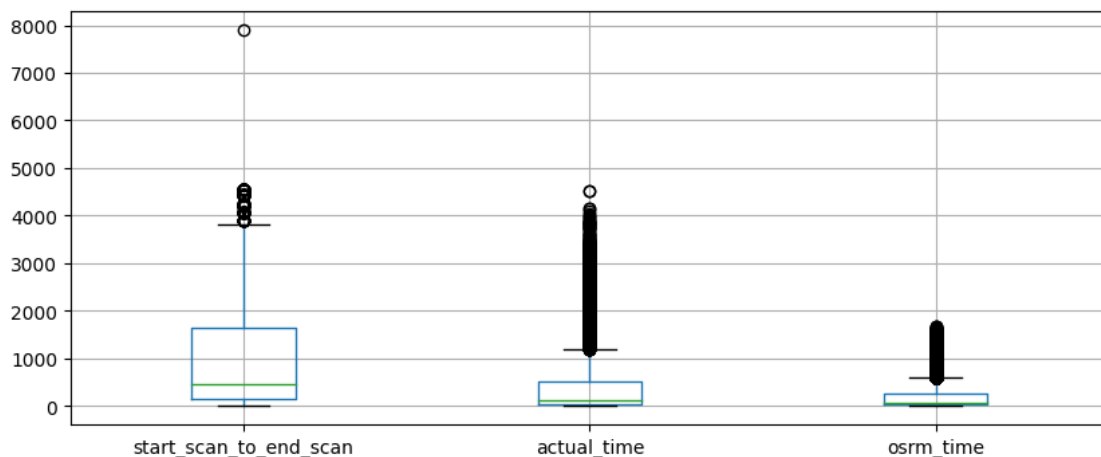
```



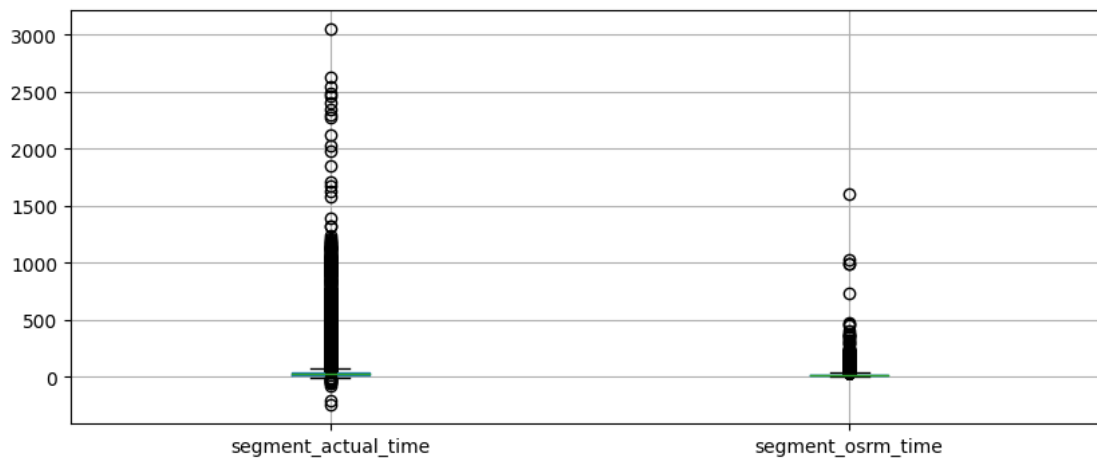
```

[ ]: df[['start_scan_to_end_scan', 'actual_time', 'osrm_time']].
      boxplot(figsize=(10,4))
plt.show()

```



```
[ ]: df[['segment_actual_time', 'segment_osrm_time']].boxplot(figsize=(10,4))
plt.show()
```



```
[ ]: # Interactive visualization

import plotly.express as px

# Interactive histogram for 'actual_time'
fig = px.histogram(df, x='actual_time', color='route_type',
                  title='Distribution of Actual Time by Route Type',
                  marginal='box', # Adds box plot on top
                  template='plotly_white')

fig.show()
```

Build some features to prepare the data for actual analysis.

Extract features from the below fields:

Destination Name: Split and extract features out of destination. City-place-code (State)

Source Name: Split and extract features out of destination. City-place-code (State)

Trip_creation_time: Extract features like month, year and day etc

```
[ ]: samp = []
for i in df['source_name'].unique():
    a = i.split('_')[0]
    if i not in samp and len(a)<4:
        samp.append(a)
print(samp)
```

```
['Una', 'MAA', 'AMD', 'BLR', 'De1', 'GZB', 'AMD', 'GGN', 'CCU', 'FBD', 'Cjb',
'Cjb', 'Goa', 'Goa', 'CCU', 'CCU', 'Amd', 'BOM', 'Pen', 'CCU', 'CCU', 'Goa',
```

```
'NOI', 'Blr', 'Goa', 'OK', 'CCU', 'Amb', 'Hyd', 'Hyd', 'Hyd', 'Mau', 'Goa',  
'Hyd', 'Wai']
```

Creating General Function for extraxting state name, city name, place name and placecode and

```
[ ]: def state_name(x):  
    l = x.split('(')  
    if len(l) == 1:  
        return 'State_'+l[0]  
    else:  
        return l[-1].replace(')', '')
```

```
[ ]: def city_name(x):  
    if 'Loc' in x:  
        return 'city_'+x  
    else:  
        l = x.split()[0].split('_')  
        if 'CCU' in x:  
            return 'Kolkata'  
        elif 'MAA' in x.upper():  
            return 'Chennai'  
        elif ('HBR' in x.upper()) or ('BLR' in x.upper()):  
            return 'Bengaluru'  
        elif 'FBD' in x.upper():  
            return 'Faridabad'  
        elif 'BOM' in x.upper():  
            return 'Mumbai'  
        elif 'DEL' in x.upper():  
            return 'Delhi'  
        elif 'OK' in x.upper():  
            return 'Delhi'  
        elif 'GZB' in x.upper():  
            return 'Ghaziabad'  
        elif 'GGN' in x.upper():  
            return 'Gurgaon'  
        elif 'AMD' in x.upper():  
            return 'Ahmedabad'  
        elif 'CJB' in x.upper():  
            return 'Coimbatore'  
        elif 'HYD' in x.upper():  
            return 'Hyderabad'  
        elif 'BBS' in x.upper():  
            return 'Bhubaneswar'  
        return l[0]
```

```
[ ]: def place_code(x):  
    if 'Loc' in x:  
        return x
```

```

elif 'HBR' in x:
    return 'HBR_PC'
else:
    try:
        if x.split('(')[0].split('_'):
            l = x.split('(')[0].split('_')
    except:
        l = ''
    if len(l) == 1:
        return 'unknown_place'
    else:
        return '_'.join(l[2:]).strip()

```

```
[ ]: df1d = pd.DataFrame(df['source_name'])
```

```
[ ]: df1d['place_code'] = df1d['source_name'].apply(place_code)
df1d
```

```
[ ]:
```

	source_name	place_code
0	Anand_VUNagar_DC (Gujarat)	DC
1	Anand_VUNagar_DC (Gujarat)	DC
2	Anand_VUNagar_DC (Gujarat)	DC
3	Anand_VUNagar_DC (Gujarat)	DC
4	Anand_VUNagar_DC (Gujarat)	DC
...	...	...
144862	Sonipat_Kundli_H (Haryana)	H
144863	Sonipat_Kundli_H (Haryana)	H
144864	Sonipat_Kundli_H (Haryana)	H
144865	Sonipat_Kundli_H (Haryana)	H
144866	Sonipat_Kundli_H (Haryana)	H

[144867 rows x 2 columns]

Since code are not unique, so using the combination of place and code as below

```
[ ]: def place_name_code(x):
    if 'Loc' in x:
        return x
    elif 'HBR' in x:
        return 'HBR_Layout_PC'
    else:
        l = x.split()[0].split('_',1)
        if len(l) == 1:
            return 'unknown_place'
        else:
            return l[1]

```

```
[ ]: df['destination_state'] = df['destination_name'].apply(state_name)
df['destination_state'].unique()
```

```
[ ]: array(['Gujarat', 'Maharashtra', 'Karnataka', 'Kerala', 'Punjab',
          'Uttarakhand', 'Tamil Nadu', 'Haryana', 'Rajasthan', 'Telangana',
          'Uttar Pradesh', 'Delhi', 'Himachal Pradesh', 'Andhra Pradesh',
          'Bihar', 'Jharkhand', 'Assam', 'Orissa', 'West Bengal',
          'Jammu & Kashmir', 'Madhya Pradesh', 'Chandigarh', 'Chhattisgarh',
          'Goa', 'Pondicherry', 'Arunachal Pradesh',
          'Dadra and Nagar Haveli', 'Meghalaya', 'Tripura', 'Mizoram',
          'Daman & Diu', 'Nagaland'], dtype=object)
```

```
[ ]: df['destination_city'] = df['destination_name'].apply(city_name)
df['destination_city'].unique()
```

```
[ ]: array(['Khambhat', 'Anand', 'Pune', ..., 'Kerala', 'AmaDubai',
          'Naraingarh'], dtype=object)
```

```
[ ]: df['destination_place'] = df['destination_name'].apply(place_name_code)
df['destination_place'].unique()
```

```
[ ]: array(['MotvdDPP_D', 'Vaghasi_IP', 'Tathawde_H', ..., 'Mylapore',
          'Ward2DPP_D', 'Ghansoli_DC'], dtype=object)
```

Source Name: Split and extract features out of destination. City-place-code (State)

```
[ ]: df['source_state'] = df['source_name'].apply(state_name)
df['source_state'].unique()
```

```
[ ]: array(['Gujarat', 'Maharashtra', 'Karnataka', 'Punjab', 'Haryana',
          'Uttarakhand', 'Tamil Nadu', 'Rajasthan', 'Telangana',
          'Madhya Pradesh', 'Uttar Pradesh', 'Himachal Pradesh', 'Kerala',
          'Andhra Pradesh', 'Bihar', 'Jharkhand', 'Assam', 'West Bengal',
          'Orissa', 'Delhi', 'Jammu & Kashmir', 'Chandigarh', 'Chhattisgarh',
          'Goa', 'Pondicherry', 'Dadra and Nagar Haveli',
          'Arunachal Pradesh', 'Nagaland', 'Meghalaya', 'Tripura', 'Mizoram'],
          dtype=object)
```

```
[ ]: df['source_city'] = df['source_name'].apply(city_name)
df['source_city'].unique()
```

```
[ ]: array(['Anand', 'Khambhat', 'Bhiwandi', ..., 'Nohar', 'Dwarka',
          'Kulithalai'], dtype=object)
```

```
[ ]: df['source_place'] = df['source_name'].apply(place_name_code)
df['source_place'].unique()
```

```
[ ]: array(['VUNagar_DC', 'MotvdDPP_D', 'Mankoli_HB', ..., 'StnRoad_DC',
          'Nelmngla_L', 'AnnaNGR_D'], dtype=object)
```

Trip_creation_time: Extract features like month, year and day etc

```
[ ]: df['trip_creation_date'] = pd.to_datetime(df['trip_creation_time'].dt.date)
df['trip_creation_date'].unique()
```

```
[ ]: <DatetimeArray>
['2018-09-20 00:00:00', '2018-09-23 00:00:00', '2018-09-14 00:00:00',
 '2018-09-13 00:00:00', '2018-09-29 00:00:00', '2018-09-17 00:00:00',
 '2018-09-12 00:00:00', '2018-10-01 00:00:00', '2018-09-27 00:00:00',
 '2018-09-28 00:00:00', '2018-09-25 00:00:00', '2018-09-15 00:00:00',
 '2018-09-18 00:00:00', '2018-09-24 00:00:00', '2018-10-03 00:00:00',
 '2018-09-19 00:00:00', '2018-09-26 00:00:00', '2018-09-22 00:00:00',
 '2018-09-21 00:00:00', '2018-10-02 00:00:00', '2018-09-16 00:00:00',
 '2018-09-30 00:00:00']
Length: 22, dtype: datetime64[ns]
```

```
[ ]: df['trip_creation_day'] = df['trip_creation_time'].dt.day
df['trip_creation_day'] = df['trip_creation_day'].astype('int')
df['trip_creation_day'].unique()
```

```
[ ]: array([20, 23, 14, 13, 29, 17, 12,  1, 27, 28, 25, 15, 18, 24,  3, 19, 26,
          22, 21,  2, 16, 30])
```

```
[ ]: df['trip_creation_month'] = df['trip_creation_time'].dt.month
df['trip_creation_month'] = df['trip_creation_month'].astype('int')
df['trip_creation_month'].unique()
```

```
[ ]: array([ 9, 10])
```

```
[ ]: df['trip_creation_year'] = df['trip_creation_time'].dt.year
df['trip_creation_year'] = df['trip_creation_year'].astype('int')
df['trip_creation_year'].unique()
```

```
[ ]: array([2018])
```

```
[ ]: df['trip_creation_week'] = df['trip_creation_time'].dt.isocalendar().week
df['trip_creation_week'] = df['trip_creation_week'].astype('int')
df['trip_creation_week'].unique()
```

```
[ ]: array([38, 37, 39, 40])
```

```
[ ]: df['trip_creation_time'] = pd.to_datetime(df['trip_creation_time'],
      ↪errors='coerce')
df['trip_creation_hour'] = df['trip_creation_time'].dt.hour
```

```
df['trip_creation_hour'] = df['trip_creation_hour'].astype('int')
df['trip_creation_hour'].unique()
```

```
[ ]: array([ 2,  6, 15, 20, 22, 21,  1, 16, 14,  8, 23, 19, 17,  0, 18,  3,  4,
          13, 11,  7,  5,  9, 12, 10])
```

The structure of data after cleaning and featuring new fields

```
[ ]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144867 entries, 0 to 144866
Data columns (total 31 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   data                                  144867 non-null  category
1   trip_creation_time                    144867 non-null  datetime64[ns]
2   route_schedule_uuid                  144867 non-null  object
3   route_type                            144867 non-null  category
4   trip_uuid                             144867 non-null  object
5   source_center                         144867 non-null  object
6   source_name                           144867 non-null  object
7   destination_center                    144867 non-null  object
8   destination_name                       144867 non-null  object
9   od_start_time                         144867 non-null  datetime64[ns]
10  od_end_time                           144867 non-null  datetime64[ns]
11  start_scan_to_end_scan                 144867 non-null  float64
12  actual_distance_to_destination          144867 non-null  float64
13  actual_time                            144867 non-null  float64
14  osrm_time                             144867 non-null  float64
15  osrm_distance                          144867 non-null  float64
16  segment_actual_time                    144867 non-null  float64
17  segment_osrm_time                      144867 non-null  float64
18  segment_osrm_distance                  144867 non-null  float64
19  destination_state                      144867 non-null  object
20  destination_city                       144867 non-null  object
21  destination_place                      144867 non-null  object
22  source_state                           144867 non-null  object
23  source_city                           144867 non-null  object
24  source_place                           144867 non-null  object
25  trip_creation_date                     144867 non-null  datetime64[ns]
26  trip_creation_day                       144867 non-null  int64
27  trip_creation_month                     144867 non-null  int64
28  trip_creation_year                     144867 non-null  int64
29  trip_creation_week                     144867 non-null  int64
30  trip_creation_hour                     144867 non-null  int64
dtypes: category(2), datetime64[ns](4), float64(8), int64(5), object(12)
memory usage: 32.3+ MB
```

```
[ ]: df.describe().T
```

```
[ ]:
count
trip_creation_time      144867  2018-09-22 13:34:23.659819264
od_start_time          144867  2018-09-22 18:02:45.855230720
od_end_time            144867  2018-09-23 10:04:31.395393024
start_scan_to_end_scan 144867.0      961.262986
actual_distance_to_destination 144867.0      234.073372
actual_time            144867.0      416.927527
osrm_time              144867.0      213.868272
osrm_distance          144867.0      284.771297
segment_actual_time    144867.0       36.196111
segment_osrm_time      144867.0      18.507548
segment_osrm_distance  144867.0       22.82902
trip_creation_date      144867  2018-09-22 00:25:08.915073792
trip_creation_day       144867.0      18.389723
trip_creation_month     144867.0       9.120925
trip_creation_year      144867.0      2018.0
trip_creation_week      144867.0      38.29991
trip_creation_hour      144867.0     12.652516
```

```
min \
trip_creation_time      2018-09-12 00:00:16.535741
od_start_time          2018-09-12 00:00:16.535741
od_end_time            2018-09-12 00:50:10.814399
start_scan_to_end_scan 20.0
actual_distance_to_destination 9.000045
actual_time            9.0
osrm_time              6.0
osrm_distance          9.0082
segment_actual_time    -244.0
segment_osrm_time      0.0
segment_osrm_distance  0.0
trip_creation_date      2018-09-12 00:00:00
trip_creation_day       1.0
trip_creation_month     9.0
trip_creation_year      2018.0
trip_creation_week      37.0
trip_creation_hour      0.0
```

```
25% \
trip_creation_time      2018-09-17 03:20:51.775845888
od_start_time          2018-09-17 08:05:40.886155008
od_end_time            2018-09-18 01:48:06.410121984
start_scan_to_end_scan 161.0
actual_distance_to_destination 23.355874
actual_time            51.0
```


osrm_time	27.0
osrm_distance	29.9147
segment_actual_time	20.0
segment_osrm_time	11.0
segment_osrm_distance	12.0701
trip_creation_date	2018-09-17 00:00:00
trip_creation_day	14.0
trip_creation_month	9.0
trip_creation_year	2018.0
trip_creation_week	38.0
trip_creation_hour	5.0

50% \

trip_creation_time	2018-09-22 04:24:27.932764928
od_start_time	2018-09-22 08:53:00.116656128
od_end_time	2018-09-23 03:13:03.520212992
start_scan_to_end_scan	449.0
actual_distance_to_destination	66.126571
actual_time	132.0
osrm_time	64.0
osrm_distance	78.5258
segment_actual_time	29.0
segment_osrm_time	17.0
segment_osrm_distance	23.513
trip_creation_date	2018-09-22 00:00:00
trip_creation_day	19.0
trip_creation_month	9.0
trip_creation_year	2018.0
trip_creation_week	38.0
trip_creation_hour	15.0

75% \

trip_creation_time	2018-09-27 17:57:56.350054912
od_start_time	2018-09-27 22:41:50.285857024
od_end_time	2018-09-28 12:49:06.054018048
start_scan_to_end_scan	1634.0
actual_distance_to_destination	286.708875
actual_time	513.0
osrm_time	257.0
osrm_distance	343.19325
segment_actual_time	40.0
segment_osrm_time	22.0
segment_osrm_distance	27.81325
trip_creation_date	2018-09-27 00:00:00
trip_creation_day	25.0
trip_creation_month	9.0
trip_creation_year	2018.0

```
trip_creation_week          39.0
trip_creation_hour          20.0
```

```

                                max      std
trip_creation_time      2018-10-03 23:59:42.701692      NaN
od_start_time           2018-10-06 04:27:23.392375      NaN
od_end_time             2018-10-08 03:00:24.353479      NaN
start_scan_to_end_scan          7898.0  1037.012769
actual_distance_to_destination    1927.447705   344.990009
actual_time                4532.0    598.103621
osrm_time                  1686.0    308.011085
osrm_distance              2326.1991   421.119294
segment_actual_time           3051.0    53.571158
segment_osrm_time            1611.0    14.77596
segment_osrm_distance         2191.4037    17.86066
trip_creation_date          2018-10-03 00:00:00      NaN
trip_creation_day           30.0     7.878764
trip_creation_month         10.0     0.326041
trip_creation_year          2018.0      0.0
trip_creation_week          40.0     0.966737
trip_creation_hour          23.0     7.949513
```

```
[ ]: df.describe(include = object).T
```

```
[ ]:
count unique \
route_schedule_uuid  144867   1504
trip_uuid            144867  14817
source_center        144867   1508
source_name          144867   1498
destination_center   144867   1481
destination_name     144867   1468
destination_state    144867    32
destination_city     144867  1196
destination_place    144867  1178
source_state         144867    31
source_city          144867  1202
source_place         144867  1207
```

```

                                top      freq
route_schedule_uuid  thanos::sroute:4029a8a2-6c74-4b7e-a6d8-f9e069f...  1812
trip_uuid            trip-153811219535896559      101
source_center        IND000000ACB      23347
source_name          Gurgaon_Bilaspur_HB (Haryana)  23640
destination_center   IND000000ACB      15192
destination_name     Gurgaon_Bilaspur_HB (Haryana)  15453
destination_state    Karnataka      21065
destination_city     Gurgaon      15886
```

destination_place	Bilaspur_HB	15453
source_state	Haryana	27792
source_city	Gurgaon	24050
source_place	Bilaspur_HB	23640

Checking correlation

Heatmap

```
[ ]: numeric_columns = df.select_dtypes(include=['number']).columns.tolist()
df_corr = df[numeric_columns].corr()
df_corr
```

```
[ ]:
start_scan_to_end_scan \
start_scan_to_end_scan      1.000000
actual_distance_to_destination 0.785006
actual_time                  0.785937
osrm_time                    0.785298
osrm_distance                0.784138
segment_actual_time          0.093301
segment_osrm_time            0.219848
segment_osrm_distance        0.306983
trip_creation_day            0.009444
trip_creation_month          0.015537
trip_creation_year           NaN
trip_creation_week           0.031197
trip_creation_hour           -0.002523
time_taken                   1.000000
```

```

actual_distance_to_destination  actual_time \
start_scan_to_end_scan          0.785006    0.785937
actual_distance_to_destination    1.000000    0.978659
actual_time                      0.978659    1.000000
osrm_time                        0.995872    0.977998
osrm_distance                    0.997149    0.979399
segment_actual_time              0.045241    0.124411
segment_osrm_time                0.158832    0.171465
segment_osrm_distance            0.232119    0.242282
trip_creation_day                0.003913    0.006397
trip_creation_month              0.013768    0.012208
trip_creation_year               NaN         NaN
trip_creation_week               0.022952    0.024402
trip_creation_hour               0.001785   -0.005165
time_taken                      0.785005    0.785936
```

```

osrm_time  osrm_distance  segment_actual_time \
start_scan_to_end_scan    0.785298    0.784138    0.093301
actual_distance_to_destination 0.995872    0.997149    0.045241
```

actual_time	0.977998	0.979399	0.124411
osrm_time	1.000000	0.999119	0.049892
osrm_distance	0.999119	1.000000	0.048705
segment_actual_time	0.049892	0.048705	1.000000
segment_osrm_time	0.177066	0.169151	0.433422
segment_osrm_distance	0.242282	0.239669	0.448959
trip_creation_day	0.004201	0.004462	0.005990
trip_creation_month	0.012349	0.012941	-0.007059
trip_creation_year	NaN	NaN	NaN
trip_creation_week	0.021242	0.022413	-0.004281
trip_creation_hour	0.000484	-0.000512	-0.003557
time_taken	0.785298	0.784137	0.093302

	segment_osrm_time	segment_osrm_distance \
start_scan_to_end_scan	0.219848	0.306983
actual_distance_to_destination	0.158832	0.232119
actual_time	0.171465	0.242282
osrm_time	0.177066	0.242282
osrm_distance	0.169151	0.239669
segment_actual_time	0.433422	0.448959
segment_osrm_time	1.000000	0.948523
segment_osrm_distance	0.948523	1.000000
trip_creation_day	0.008229	0.008242
trip_creation_month	-0.011078	-0.008763
trip_creation_year	NaN	NaN
trip_creation_week	-0.009242	-0.006010
trip_creation_hour	0.013366	0.010524
time_taken	0.219846	0.306980

	trip_creation_day	trip_creation_month \
start_scan_to_end_scan	0.009444	0.015537
actual_distance_to_destination	0.003913	0.013768
actual_time	0.006397	0.012208
osrm_time	0.004201	0.012349
osrm_distance	0.004462	0.012941
segment_actual_time	0.005990	-0.007059
segment_osrm_time	0.008229	-0.011078
segment_osrm_distance	0.008242	-0.008763
trip_creation_day	1.000000	-0.771211
trip_creation_month	-0.771211	1.000000
trip_creation_year	NaN	NaN
trip_creation_week	-0.050817	0.652244
trip_creation_hour	-0.002552	0.005335
time_taken	0.009443	0.015538

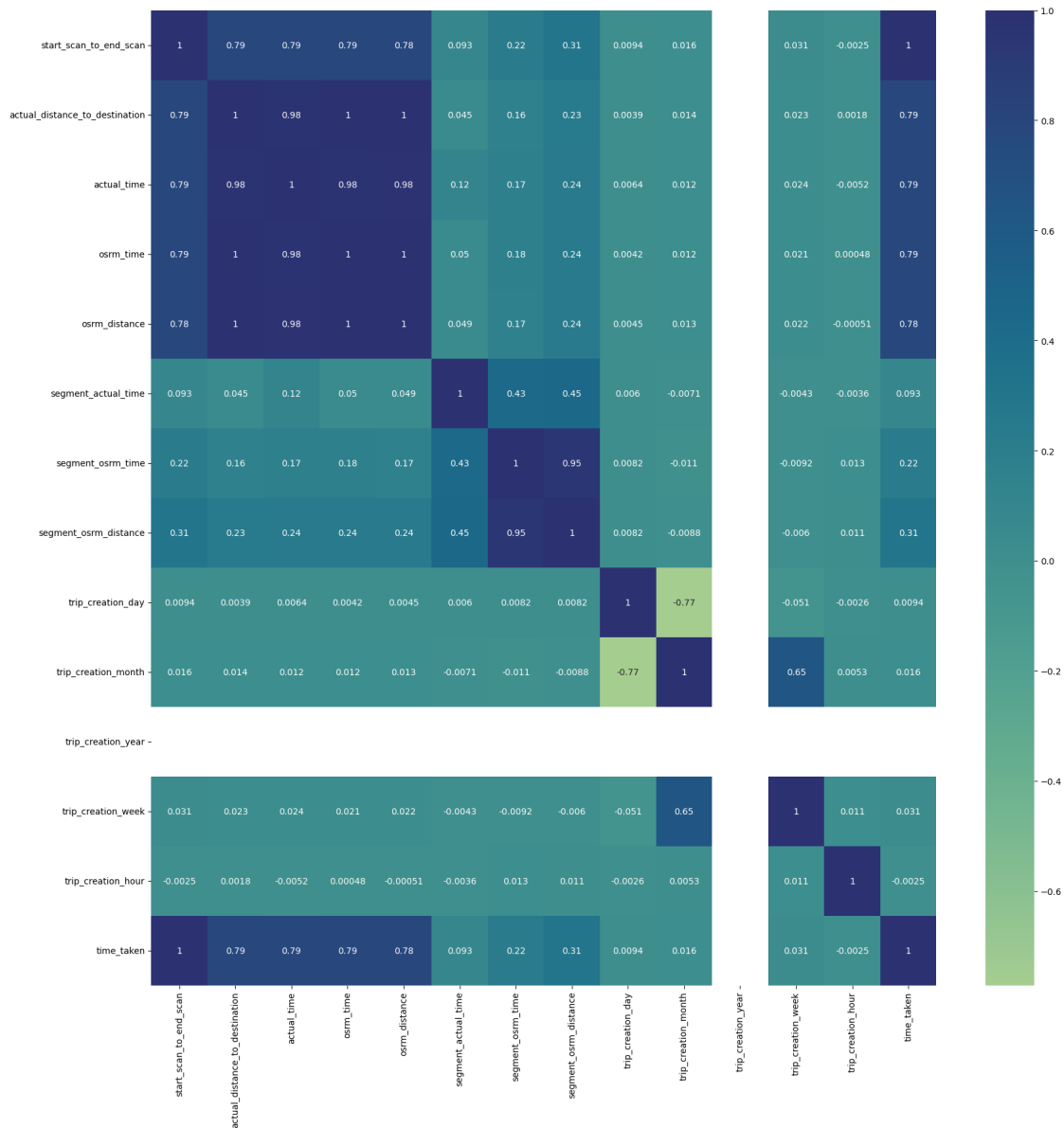
	trip_creation_year	trip_creation_week \
start_scan_to_end_scan	NaN	0.031197

actual_distance_to_destination	NaN	0.022952
actual_time	NaN	0.024402
osrm_time	NaN	0.021242
osrm_distance	NaN	0.022413
segment_actual_time	NaN	-0.004281
segment_osrm_time	NaN	-0.009242
segment_osrm_distance	NaN	-0.006010
trip_creation_day	NaN	-0.050817
trip_creation_month	NaN	0.652244
trip_creation_year	NaN	NaN
trip_creation_week	NaN	1.000000
trip_creation_hour	NaN	0.011397
time_taken	NaN	0.031196

	trip_creation_hour	time_taken
start_scan_to_end_scan	-0.002523	1.000000
actual_distance_to_destination	0.001785	0.785005
actual_time	-0.005165	0.785936
osrm_time	0.000484	0.785298
osrm_distance	-0.000512	0.784137
segment_actual_time	-0.003557	0.093302
segment_osrm_time	0.013366	0.219846
segment_osrm_distance	0.010524	0.306980
trip_creation_day	-0.002552	0.009443
trip_creation_month	0.005335	0.015538
trip_creation_year	NaN	NaN
trip_creation_week	0.011397	0.031196
trip_creation_hour	1.000000	-0.002527
time_taken	-0.002527	1.000000

```
[ ]: plt.figure(figsize = (20, 20))
sns.heatmap(data = df_corr, annot = True, cmap="crest")
plt.plot()
```

```
[ ]: []
```



In-depth analysis and feature engineering

Calculate the time taken between `od_start_time` and `od_end_time` and keep it as a feature. Drop the original columns, if required

```
[ ]: # Convert to datetime if not already in datetime format
df['od_start_time'] = pd.to_datetime(df['od_start_time'])
df['od_end_time'] = pd.to_datetime(df['od_end_time'])
df['od_total_time'] = (df['od_end_time'] - df['od_start_time']).dt.
    ↪total_seconds()
print(df[['od_start_time', 'od_end_time', 'od_total_time']].head())
```

```
df[['od_total_time', 'start_scan_to_end_scan']].describe().T
```

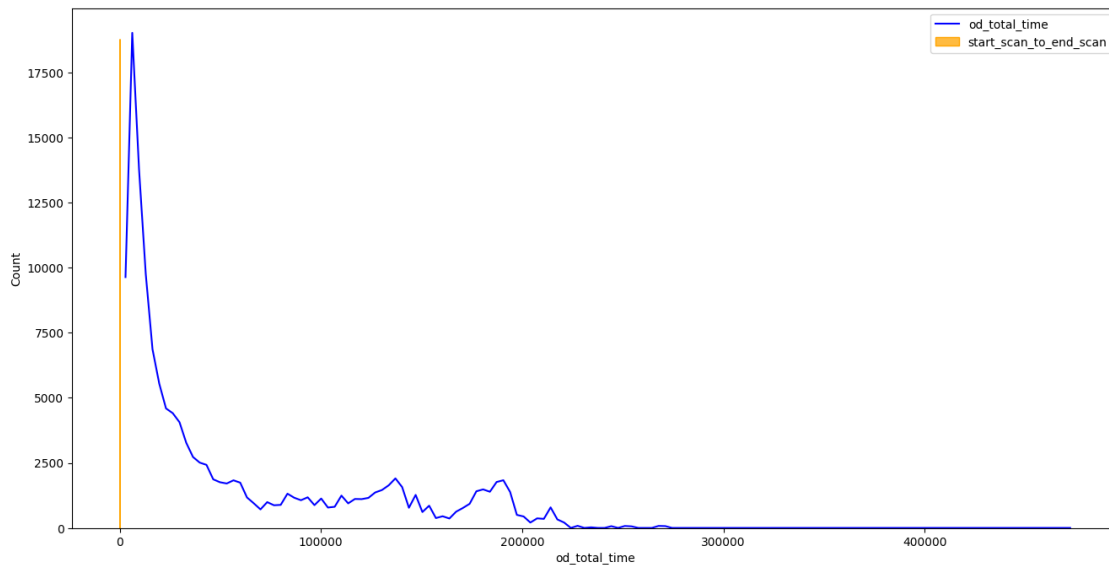
	od_start_time	od_end_time	od_total_time
0	2018-09-20 03:21:32.418600	2018-09-20 04:47:45.236797	5172.818197
1	2018-09-20 03:21:32.418600	2018-09-20 04:47:45.236797	5172.818197
2	2018-09-20 03:21:32.418600	2018-09-20 04:47:45.236797	5172.818197
3	2018-09-20 03:21:32.418600	2018-09-20 04:47:45.236797	5172.818197
4	2018-09-20 03:21:32.418600	2018-09-20 04:47:45.236797	5172.818197

```
[ ]:
count      mean      std      min  \
od_total_time    144867.0  5.770554e+04  62220.756769  1242.168787
start_scan_to_end_scan  144867.0 -2.483787e-16    1.000003   -0.909664

      25%      50%      75%      max
od_total_time    9690.077518  26954.194038  98097.008596  473913.117274
start_scan_to_end_scan   -0.773231   -0.494559    0.652060    2.789996
```

```
[ ]: plt.figure(figsize = (16, 8))
sns.histplot(df['od_total_time'], color = 'blue', fill=False, element="poly")
sns.histplot(df['start_scan_to_end_scan'], color = 'orange', element="poly")
plt.legend(['od_total_time', 'start_scan_to_end_scan'])
plt.plot()
```

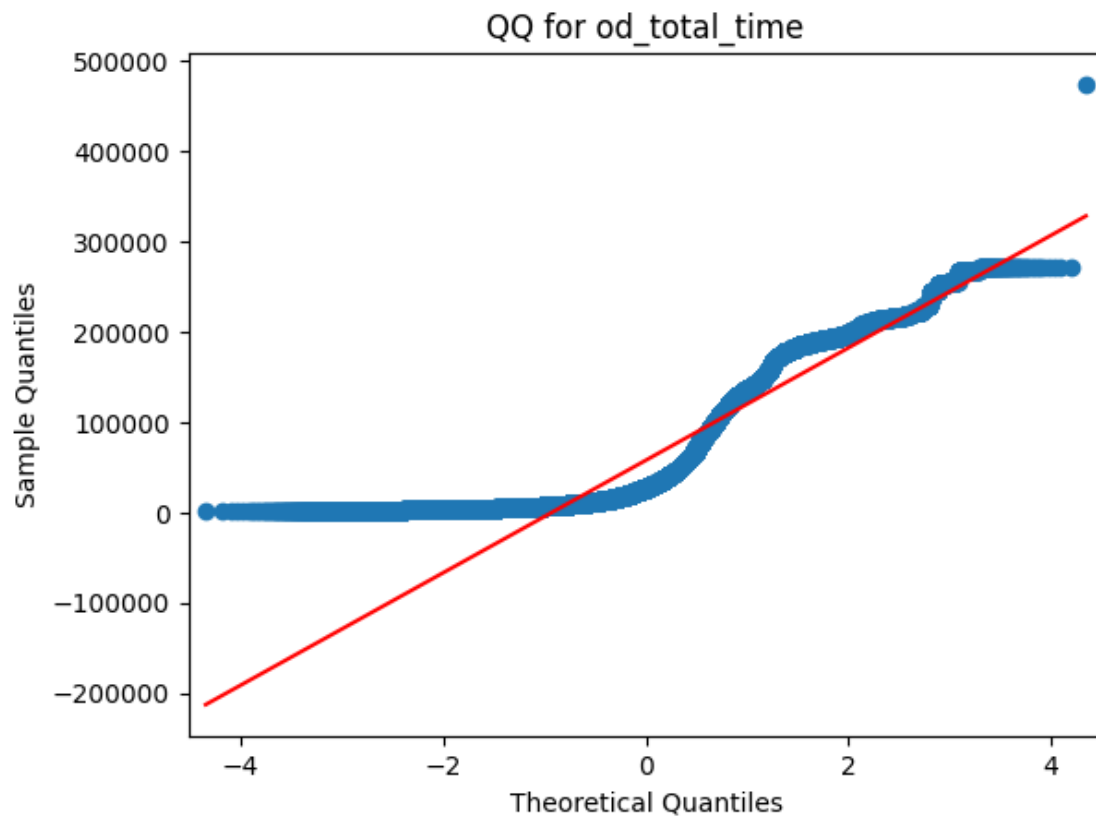
```
[ ]: [ ]
```

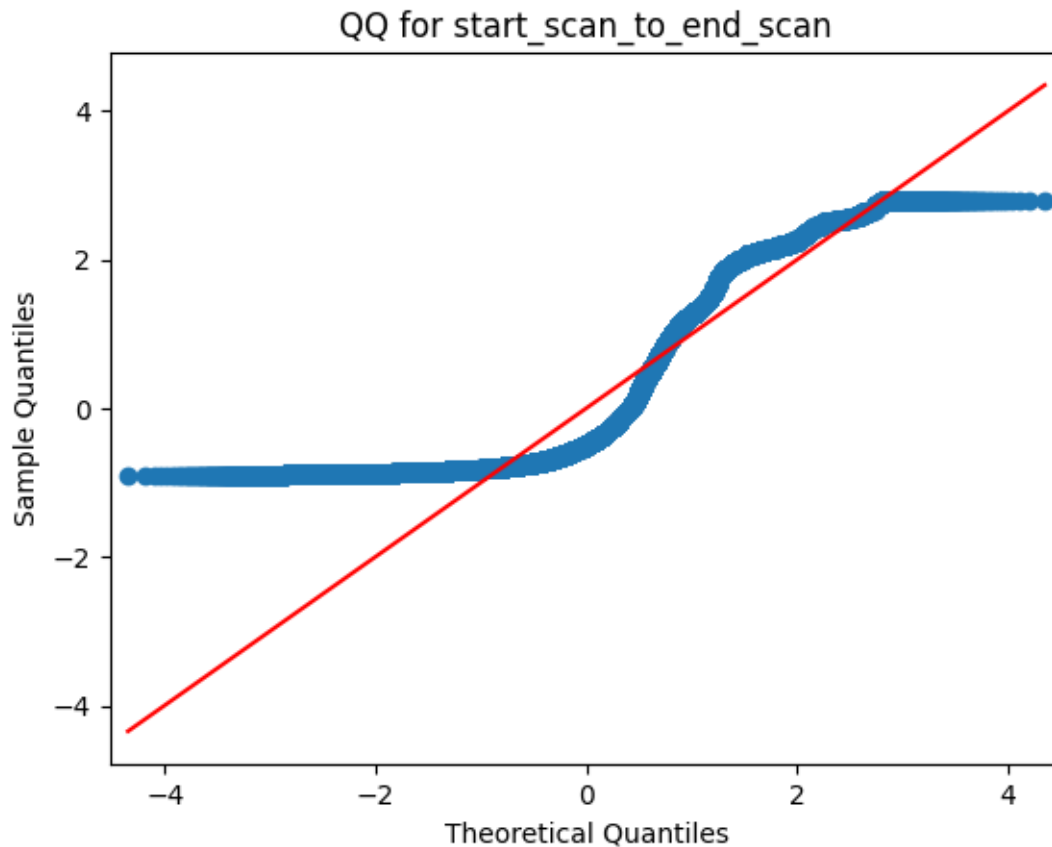


```
[ ]: # Distribution check: QQ Plot
sm.qqplot(df['od_total_time'], line='s')
plt.title('QQ for od_total_time')
```

```
plt.show()

sm.qqplot(df['start_scan_to_end_scan'],line='s')
plt.title('QQ for start_scan_to_end_scan')
plt.show()
```





It can be observed from the above plots that the samples are not normally distributed.

Applying Shapiro-Wilk test for normality

H_0 : The sample follows normal distribution

H_a : The sample does not follow normal distribution

```
[ ]: shap_stat, p_val = shapiro(df['od_total_time'])

print(f"shapiro_stat : {shap_stat}")
print(f"p-value : {p_val}")
print()
if p_val < 0.05:
    print('Reject H0: Data does not follow normal distribution')
else:
    print('Fail to reject H0: Data follows a normal distribution')
```

shapiro_stat : 0.8046415904483419

p-value : 3.935444851373457e-140

Reject H_0 : Data does not follow normal distribution

/usr/local/lib/python3.10/dist-packages/scipy/stats/_axis_nan_policy.py:531:
UserWarning:

scipy.stats.shapiro: For N > 5000, computed p-value may not be accurate. Current
N is 144867.

```
[ ]: shap_stat, p_val = shapiro(df['start_scan_to_end_scan'])

print(f"shapiro_stat : {shap_stat}")
print(f"p-value : {p_val}")
print()
if p_val < 0.05:
    print('Reject H0: Data does not follow normal distribution')
else:
    print('Fail to reject H0: Data follows a normal distribution')
```

shapiro_stat : 0.8037152322330507
p-value : 2.812900596661821e-140

Reject H0: Data does not follow normal distribution

/usr/local/lib/python3.10/dist-packages/scipy/stats/_axis_nan_policy.py:531:
UserWarning:

scipy.stats.shapiro: For N > 5000, computed p-value may not be accurate. Current
N is 144867.

BOX-COX Transformation

Using boxcox transformation to check, if the transformed data follows normal distribution.

```
[ ]: # Check for non-positive values in start_scan_to_end_scan
if (df['start_scan_to_end_scan'] <= 0).any():
    print("Non-positive values found in 'start_scan_to_end_scan'. Adjusting
    ↪data...")
    # Option 1: Filter out non-positive values
    filtered_df = df[df['start_scan_to_end_scan'] > 0]

    # Option 2: Shift values to make them positive (if you want to keep all
    ↪data)
    # min_value = df['start_scan_to_end_scan'].min()
    # df['start_scan_to_end_scan'] += abs(min_value) + 1 # Shift to make all
    ↪positive

    # Perform Box-Cox transformation on the filtered data
    od_total_time_trans, lam1 = stats.boxcox(df['od_total_time'].dropna())
```

```

    start_scan_to_end_scan_trans, lam2 = stats.
↳boxcox(filtered_df['start_scan_to_end_scan'].dropna())

else:
    # Perform Box-Cox transformation without filtering
    od_total_time_trans, lam1 = stats.boxcox(df['od_total_time'].dropna())
    start_scan_to_end_scan_trans, lam2 = stats.
↳boxcox(df['start_scan_to_end_scan'].dropna())

# Shapiro-Wilk test for the transformed od_total_time
shap_stat, p_val = shapiro(od_total_time_trans)
print(f"Shapiro Statistic (od_total_time): {shap_stat}")
print(f"p-value: {p_val}\n")
if p_val < 0.05:
    print('Reject H0: Data does not follow a normal distribution')
else:
    print('Fail to reject H0: Data follows a normal distribution')

# Shapiro-Wilk test for the transformed start_scan_to_end_scan
shap_stat, p_val = shapiro(start_scan_to_end_scan_trans)
print(f"Shapiro Statistic (start_scan_to_end_scan): {shap_stat}")
print(f"p-value: {p_val}\n")
if p_val < 0.05:
    print('Reject H0: Data does not follow a normal distribution')
else:
    print('Fail to reject H0: Data follows a normal distribution')

```

Non-positive values found in 'start_scan_to_end_scan'. Adjusting data...

Shapiro Statistic (od_total_time): 0.9493794881862714

p-value: 1.217365002785175e-101

Reject H0: Data does not follow a normal distribution

Shapiro Statistic (start_scan_to_end_scan): 0.9657159359788896

p-value: 1.2059321681752535e-71

Reject H0: Data does not follow a normal distribution

/usr/local/lib/python3.10/dist-packages/scipy/stats/_axis_nan_policy.py:531:

UserWarning:

scipy.stats.shapiro: For N > 5000, computed p-value may not be accurate. Current N is 49326.

Key Observation:

After applying the boxcox transformation on “od_total_time” and “start_scan_to_end_scan” columns, distributions do not follow normal distribution.

Homogeneity of Variances using Lavene's test

```
[ ]: lev_stat, p_vala = levene(df['od_total_time'], df['start_scan_to_end_scan'])

print(f"levene_stat : {lev_stat}")
print(f"p-value : {p_vala}")
print()
if p_vala < 0.05:
    print('Reject H0: The samples do not have Homogenous Variance')
else:
    print('Fail to reject H0: The samples have Homogenous Variance')
```

levene_stat : 117070.5680667251

p-value : 0.0

Reject H0: The samples do not have Homogenous Variance

```
[ ]: # Convert od_start_time and od_end_time to datetime
df['od_start_time'] = pd.to_datetime(df['od_start_time'], errors='coerce')
df['od_end_time'] = pd.to_datetime(df['od_end_time'], errors='coerce')

# Check for any conversion issues
if df['od_start_time'].isnull().any() or df['od_end_time'].isnull().any():
    print("Some times couldn't be converted. Check the input format.")

# Calculate time taken between od_start_time and od_end_time
df['time_taken'] = (df['od_end_time'] - df['od_start_time']).dt.total_seconds()
    ↪ # in seconds

# Optionally, convert to minutes or hours
# df['time_taken_minutes'] = df['time_taken'] / 60
# df['time_taken_hours'] = df['time_taken'] / 3600

# Display the resulting DataFrame
print(df)
```

	data	trip_creation_time \
0	training	2018-09-20 02:35:36.476840
1	training	2018-09-20 02:35:36.476840
2	training	2018-09-20 02:35:36.476840
3	training	2018-09-20 02:35:36.476840
4	training	2018-09-20 02:35:36.476840
...	...	...
144862	training	2018-09-20 16:24:28.436231
144863	training	2018-09-20 16:24:28.436231
144864	training	2018-09-20 16:24:28.436231
144865	training	2018-09-20 16:24:28.436231
144866	training	2018-09-20 16:24:28.436231

		route_schedule_uuid	route_type	\
0	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...		Carting	
1	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...		Carting	
2	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...		Carting	
3	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...		Carting	
4	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...		Carting	
...		...	...	
144862	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...		Carting	
144863	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...		Carting	
144864	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...		Carting	
144865	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...		Carting	
144866	thanos::sroute:f0569d2f-4e20-4c31-8542-67b86d5...		Carting	

		trip_uuid	source_center	source_name	\
0	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC	(Gujarat)	
1	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC	(Gujarat)	
2	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC	(Gujarat)	
3	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC	(Gujarat)	
4	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC	(Gujarat)	
...		...	...	...	
144862	trip-153746066843555182	IND131028AAB	Sonipat_Kundli_H	(Haryana)	
144863	trip-153746066843555182	IND131028AAB	Sonipat_Kundli_H	(Haryana)	
144864	trip-153746066843555182	IND131028AAB	Sonipat_Kundli_H	(Haryana)	
144865	trip-153746066843555182	IND131028AAB	Sonipat_Kundli_H	(Haryana)	
144866	trip-153746066843555182	IND131028AAB	Sonipat_Kundli_H	(Haryana)	

		destination_center	destination_name	\
0	IND388620AAB	Khambhat_MotvdDPP_D	(Gujarat)	
1	IND388620AAB	Khambhat_MotvdDPP_D	(Gujarat)	
2	IND388620AAB	Khambhat_MotvdDPP_D	(Gujarat)	
3	IND388620AAB	Khambhat_MotvdDPP_D	(Gujarat)	
4	IND388620AAB	Khambhat_MotvdDPP_D	(Gujarat)	
...		...	...	
144862	IND000000ACB	Gurgaon_Bilaspur_HB	(Haryana)	
144863	IND000000ACB	Gurgaon_Bilaspur_HB	(Haryana)	
144864	IND000000ACB	Gurgaon_Bilaspur_HB	(Haryana)	
144865	IND000000ACB	Gurgaon_Bilaspur_HB	(Haryana)	
144866	IND000000ACB	Gurgaon_Bilaspur_HB	(Haryana)	

		od_start_time	...	source_state	source_city	\
0	2018-09-20	03:21:32.418600	...	Gujarat	Anand	
1	2018-09-20	03:21:32.418600	...	Gujarat	Anand	
2	2018-09-20	03:21:32.418600	...	Gujarat	Anand	
3	2018-09-20	03:21:32.418600	...	Gujarat	Anand	
4	2018-09-20	03:21:32.418600	...	Gujarat	Anand	
...		...	...	...	...	
144862	2018-09-20	16:24:28.436231	...	Haryana	Sonipat	

144863	2018-09-20	16:24:28.436231	...	Haryana	Sonipat
144864	2018-09-20	16:24:28.436231	...	Haryana	Sonipat
144865	2018-09-20	16:24:28.436231	...	Haryana	Sonipat
144866	2018-09-20	16:24:28.436231	...	Haryana	Sonipat

	source_place	trip_creation_date	trip_creation_day	\
0	VUNagar_DC	2018-09-20	20	
1	VUNagar_DC	2018-09-20	20	
2	VUNagar_DC	2018-09-20	20	
3	VUNagar_DC	2018-09-20	20	
4	VUNagar_DC	2018-09-20	20	
...	...	...	...	
144862	Kundli_H	2018-09-20	20	
144863	Kundli_H	2018-09-20	20	
144864	Kundli_H	2018-09-20	20	
144865	Kundli_H	2018-09-20	20	
144866	Kundli_H	2018-09-20	20	

	trip_creation_month	trip_creation_year	trip_creation_week	\
0	9	2018	38	
1	9	2018	38	
2	9	2018	38	
3	9	2018	38	
4	9	2018	38	
...	...	...	...	
144862	9	2018	38	
144863	9	2018	38	
144864	9	2018	38	
144865	9	2018	38	
144866	9	2018	38	

	trip_creation_hour	time_taken
0	2	5172.818197
1	2	5172.818197
2	2	5172.818197
3	2	5172.818197
4	2	5172.818197
...	...	...
144862	16	25661.181838
144863	16	25661.181838
144864	16	25661.181838
144865	16	25661.181838
144866	16	25661.181838

[144867 rows x 32 columns]

```
[ ]: # Efficiently calculate od_total_time using vectorized operations
df['od_total_time'] = df['time_taken'].fillna(0) + df['start_scan_to_end_scan'].
    ↪ fillna(0)

# Define numeric columns for pairplot
num_col = [
    'od_total_time',
    'start_scan_to_end_scan',
    'actual_distance_to_destination',
    'actual_time',
    'osrm_time',
    'osrm_distance',
    'segment_actual_time',
    'segment_osrm_time',
    'segment_osrm_distance'
]

# Sample the data to speed up visualization if the dataset is large
sample_size = 1000 # Adjust this number based on your dataset size
if len(df) > sample_size:
    df_sample = df.sample(sample_size, random_state=1)
else:
    df_sample = df

# Create the pairplot
sns.pairplot(data=df_sample, vars=num_col, kind='reg', hue='route_type')
plt.show()
```

/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning:

When grouping with a length-1 list-like, you will need to pass a length-1 tuple to get_group in a future version of pandas. Pass `(name,)` instead of `name` to silence this warning.

/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning:

When grouping with a length-1 list-like, you will need to pass a length-1 tuple to get_group in a future version of pandas. Pass `(name,)` instead of `name` to silence this warning.

/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning:

When grouping with a length-1 list-like, you will need to pass a length-1 tuple to get_group in a future version of pandas. Pass `(name,)` instead of `name` to silence this warning.

/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning:

When grouping with a length-1 list-like, you will need to pass a length-1 tuple to `get_group` in a future version of pandas. Pass ``(name,)`` instead of ``name`` to silence this warning.

```
/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning:
```

When grouping with a length-1 list-like, you will need to pass a length-1 tuple to `get_group` in a future version of pandas. Pass ``(name,)`` instead of ``name`` to silence this warning.

```
/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning:
```

When grouping with a length-1 list-like, you will need to pass a length-1 tuple to `get_group` in a future version of pandas. Pass ``(name,)`` instead of ``name`` to silence this warning.

```
/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning:
```

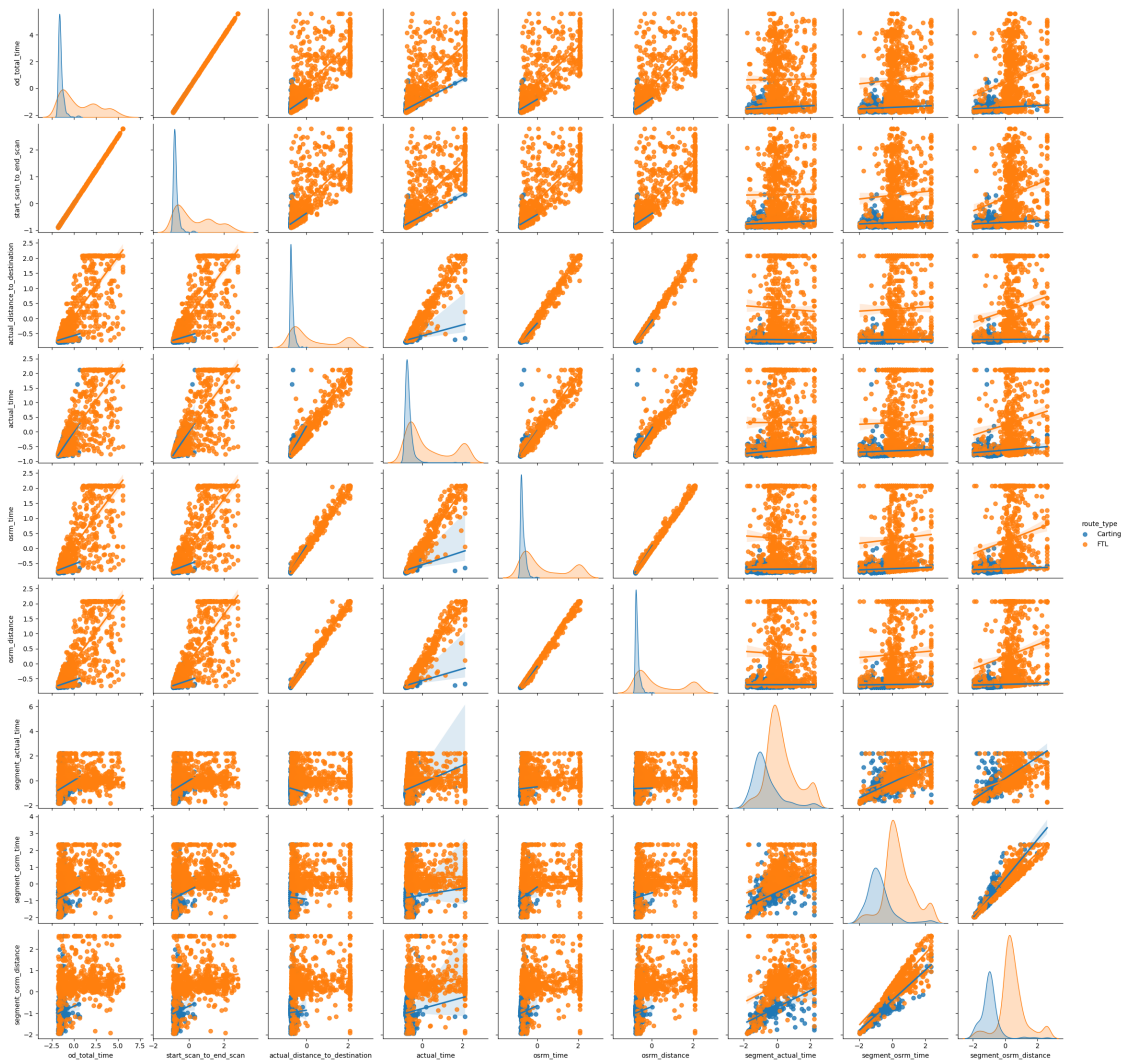
When grouping with a length-1 list-like, you will need to pass a length-1 tuple to `get_group` in a future version of pandas. Pass ``(name,)`` instead of ``name`` to silence this warning.

```
/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning:
```

When grouping with a length-1 list-like, you will need to pass a length-1 tuple to `get_group` in a future version of pandas. Pass ``(name,)`` instead of ``name`` to silence this warning.

```
/usr/local/lib/python3.10/dist-packages/seaborn/_base.py:949: FutureWarning:
```

When grouping with a length-1 list-like, you will need to pass a length-1 tuple to `get_group` in a future version of pandas. Pass ``(name,)`` instead of ``name`` to silence this warning.



Compare the difference between Point a. and start_scan_to_end_scan. Do hypothesis testing/
Visual analysis to check.

```
[ ]: # Perform a paired t-test
t_stat, p_value = stats.ttest_rel(df['time_taken'],
    ↪df['start_scan_to_end_scan'])

# Output the results
print(f"T-statistic: {t_stat}")
print(f"P-value: {p_value}")

# Interpret the p-value
alpha = 0.05
if p_value < alpha:
```

```

print("Reject the null hypothesis: There is a significant difference_
↪between the two time measurements.")
else:
    print("Fail to reject the null hypothesis: There is no significant_
↪difference between the two time measurements.")

```

T-statistic: 352.9967867233228

P-value: 0.0

Reject the null hypothesis: There is a significant difference between the two time measurements.

```

[ ]: plt.figure(figsize=(12, 6))

# Melt the DataFrame to use in seaborn
melted_df = df[['time_taken', 'start_scan_to_end_scan']].
    ↪melt(var_name='Time_Type', value_name='Time_Seconds')

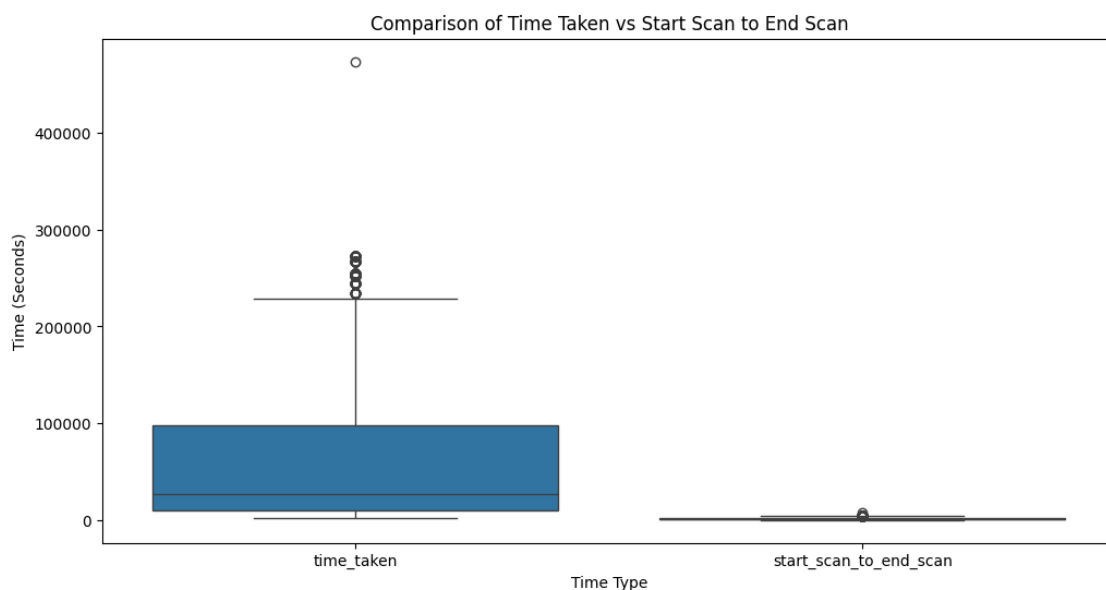
# Create a box plot to compare the two time features
sns.boxplot(x='Time_Type', y='Time_Seconds', data=melted_df)
plt.title('Comparison of Time Taken vs Start Scan to End Scan')
plt.xlabel('Time Type')
plt.ylabel('Time (Seconds)')
plt.show()

```

/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:

FutureWarning:

SeriesGroupBy.grouper is deprecated and will be removed in a future version of pandas.



Do hypothesis testing/ visual analysis between actual_time aggregated value and OSRM time aggregated value (aggregated values are the values you'll get after merging the rows on the basis of trip_uuid)

```
[ ]: # Visual analysis

# Create a new DataFrame with only the relevant columns for melting
comparison_df = df[['actual_time', 'osrm_time']]

# Melt the DataFrame for easier plotting
melted_agg_df = comparison_df.melt(var_name='Time_Type',
    ↪value_name='Time_Seconds')

# Check the melted DataFrame
print(melted_agg_df.head())

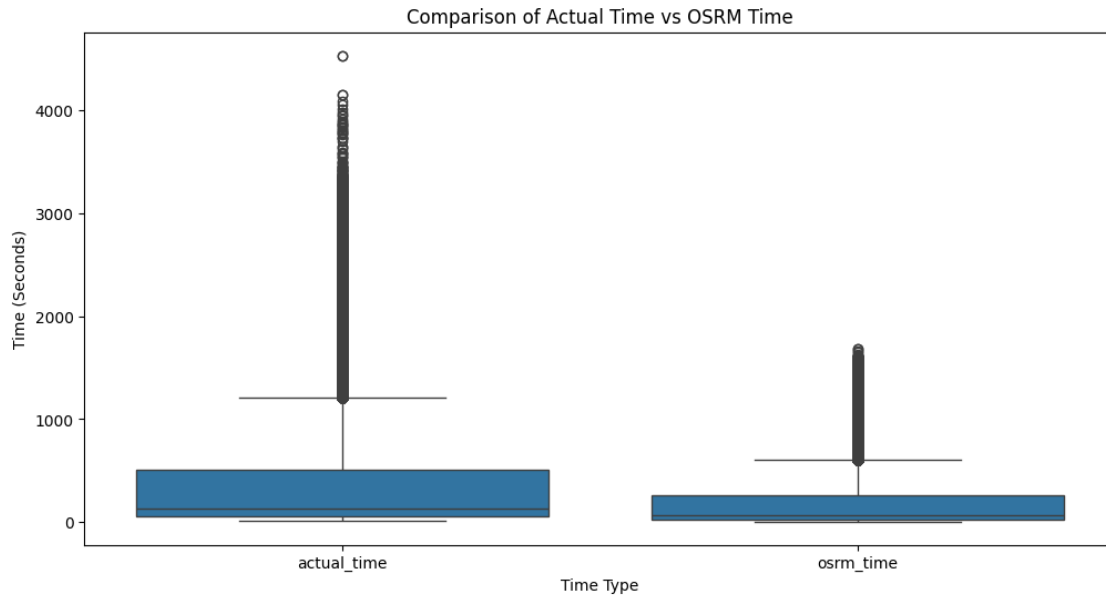
# Set up the figure
plt.figure(figsize=(12, 6))

# Create a box plot to compare the two aggregated time features
sns.boxplot(x='Time_Type', y='Time_Seconds', data=melted_agg_df)
plt.title('Comparison of Actual Time vs OSRM Time')
plt.xlabel('Time Type')
plt.ylabel('Time (Seconds)')
plt.show()
```

	Time_Type	Time_Seconds
0	actual_time	14.0
1	actual_time	24.0
2	actual_time	40.0
3	actual_time	62.0
4	actual_time	68.0

```
/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:
FutureWarning:
```

SeriesGroupBy.grouper is deprecated and will be removed in a future version of pandas.



```
[ ]: # Perform a paired t-test
t_stat, p_value = stats.ttest_rel(df['actual_time'], df['osrm_time'])

# Output the results
print(f"T-statistic: {t_stat}")
print(f"P-value: {p_value}")

# Interpret the p-value
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: There is a significant difference_
    ↪ between actual time and OSRM time.")
else:
    print("Fail to reject the null hypothesis: There is no significant_
    ↪ difference between actual time and OSRM time.")
```

T-statistic: 254.44873336004324

P-value: 0.0

Reject the null hypothesis: There is a significant difference between actual time and OSRM time.

Do hypothesis testing/ visual analysis between actual_time aggregated value and segment actual time aggregated value (aggregated values are the values you'll get after merging the rows on the basis of trip_uuid)

```
[ ]: # Aggregate actual_time and segment_actual_time based on trip_uuid
aggregated_df = df.groupby('trip_uuid').agg({
    'actual_time': 'sum', # or 'mean', depending on your analysis
```

```

    'segment_actual_time': 'sum' # or 'mean'
}).reset_index()

```

```

[ ]: # Visual analysis

plt.figure(figsize=(12, 6))

comparison_df = df[['actual_time', 'segment_actual_time']]

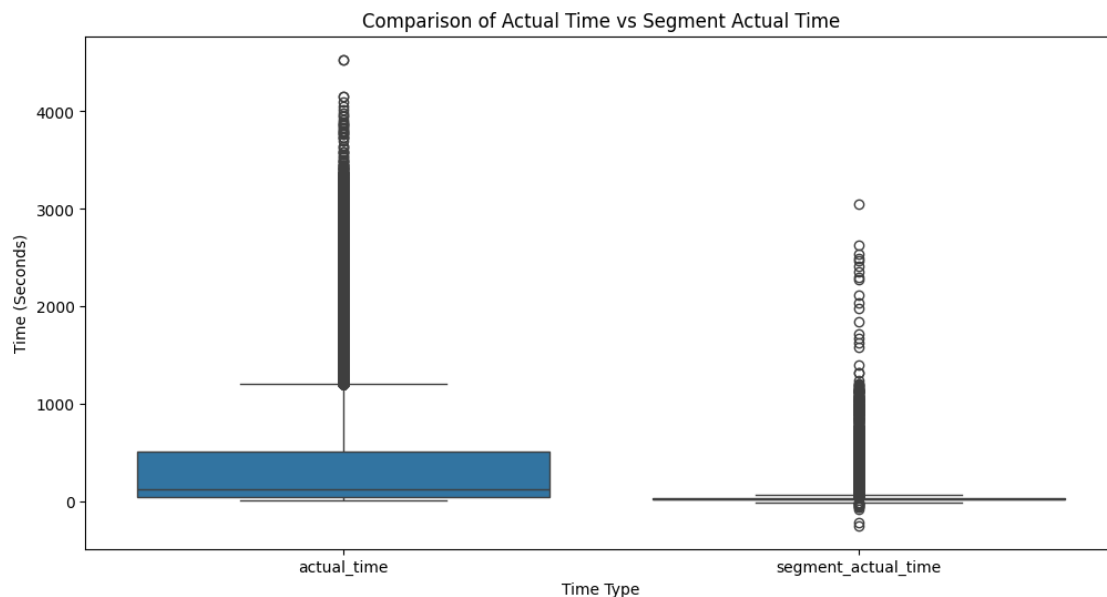
# Melt the DataFrame for easier plotting
melted_agg_df = comparison_df.melt(var_name='Time_Type',
    value_name='Time_Seconds')

# Create a box plot to compare the two aggregated time features
sns.boxplot(x='Time_Type', y='Time_Seconds', data=melted_agg_df)
plt.title('Comparison of Actual Time vs Segment Actual Time')
plt.xlabel('Time Type')
plt.ylabel('Time (Seconds)')
plt.show()

```

/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:
FutureWarning:

SeriesGroupBy.grouper is deprecated and will be removed in a future version of pandas.



```
[ ]: t_stat, p_value = stats.ttest_rel(aggregated_df['actual_time'],
    ↪ aggregated_df['segment_actual_time'])

# Output the results
print(f"T-statistic: {t_stat}")
print(f"P-value: {p_value}")

# Interpret the p-value
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: There is a significant difference
    ↪ between actual time and segment actual time.")
else:
    print("Fail to reject the null hypothesis: There is no significant
    ↪ difference between actual time and segment actual time.")
```

T-statistic: 30.75550616001704

P-value: 2.077325421800874e-201

Reject the null hypothesis: There is a significant difference between actual time and segment actual time.

Do hypothesis testing/ visual analysis between osrm distance aggregated value and segment osrm distance aggregated value (aggregated values are the values you'll get after merging the rows on the basis of trip_uuid)

```
[ ]: # Aggregate osrm_distance and segment_osrm_distance based on trip_uuid
aggregated_df = df.groupby('trip_uuid').agg({
    'osrm_distance': 'sum', # or 'mean', depending on your analysis
    'segment_osrm_distance': 'sum' # or 'mean'
}).reset_index()
```

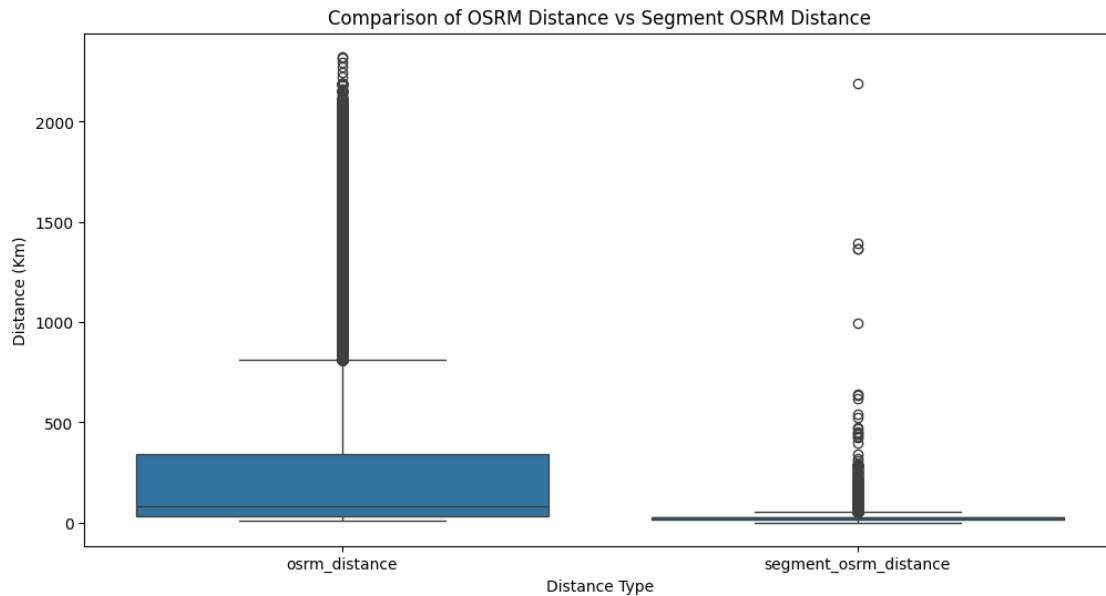
```
[ ]: # Visual analysis
plt.figure(figsize=(12, 6))

# Melt the DataFrame for easier plotting
comparison_df = df[['osrm_distance', 'segment_osrm_distance']]
melted_agg_df = comparison_df.melt(var_name='Distance_Type',
    ↪ value_name='Distance_Km')

# Create a box plot to compare the two aggregated distance features
sns.boxplot(x='Distance_Type', y='Distance_Km', data=melted_agg_df)
plt.title('Comparison of OSRM Distance vs Segment OSRM Distance')
plt.xlabel('Distance Type')
plt.ylabel('Distance (Km)')
plt.show()
```

/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:
FutureWarning:

SeriesGroupBy.grouper is deprecated and will be removed in a future version of pandas.



```
[ ]: t_stat, p_value = stats.ttest_rel(aggregated_df['osrm_distance'],
    ↪ aggregated_df['segment_osrm_distance'])

# Output the results
print(f"T-statistic: {t_stat}")
print(f"P-value: {p_value}")

# Interpret the p-value
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: There is a significant difference
    ↪ between OSRM distance and segment OSRM distance.")
else:
    print("Fail to reject the null hypothesis: There is no significant
    ↪ difference between OSRM distance and segment OSRM distance.")
```

T-statistic: 30.03031541377046

P-value: 2.1753879024067997e-192

Reject the null hypothesis: There is a significant difference between OSRM distance and segment OSRM distance.

Do hypothesis testing/ visual analysis between osrm time aggregated value and segment osrm time aggregated value (aggregated values are the values you'll get after merging the rows on the basis

of trip_uuid)

```
[ ]: # Aggregate osrm_time and segment_osrm_time based on trip_uuid
aggregated_df = df.groupby('trip_uuid').agg({
    'osrm_time': 'sum', # or 'mean', depending on your analysis
    'segment_osrm_time': 'sum' # or 'mean'
}).reset_index()
```

```
[ ]: # Visual analysis

plt.figure(figsize=(12, 6))

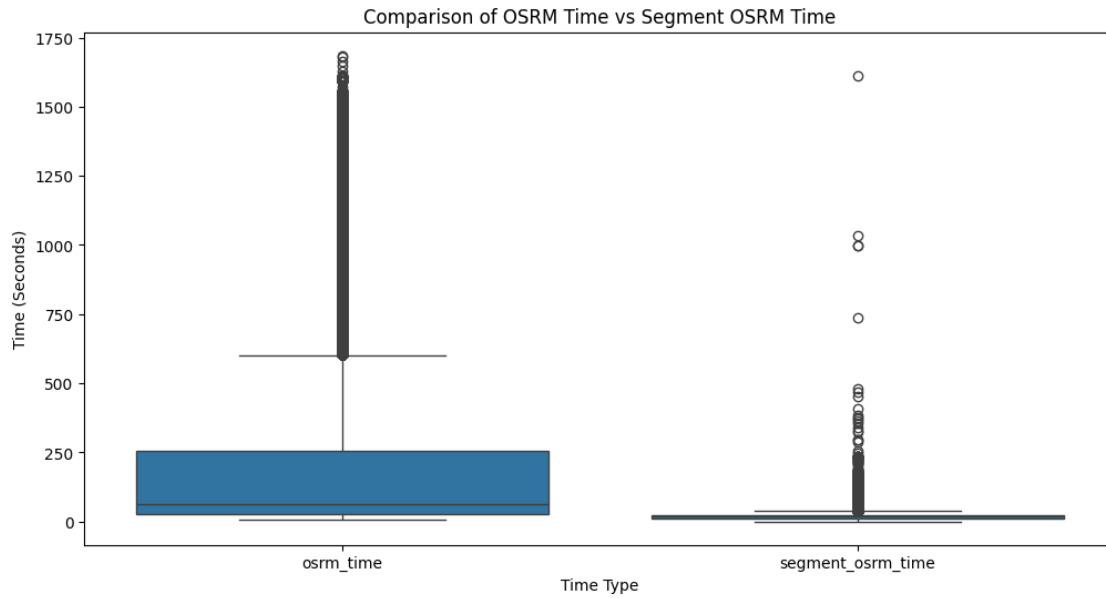
comparison_df = df[['osrm_time', 'segment_osrm_time']]

# Melt the DataFrame for easier plotting
melted_agg_df = comparison_df.melt(var_name='Time_Type',
    value_name='Time_Seconds')

# Create a box plot to compare the two aggregated time features
sns.boxplot(x='Time_Type', y='Time_Seconds', data=melted_agg_df)
plt.title('Comparison of OSRM Time vs Segment OSRM Time')
plt.xlabel('Time Type')
plt.ylabel('Time (Seconds)')
plt.show()
```

/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:
FutureWarning:

SeriesGroupBy.grouper is deprecated and will be removed in a future version of pandas.



```
[ ]: t_stat, p_value = stats.ttest_rel(aggregated_df['osrm_time'],
    ↪ aggregated_df['segment_osrm_time'])

# Output the results
print(f"T-statistic: {t_stat}")
print(f"P-value: {p_value}")

# Interpret the p-value
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: There is a significant difference
    ↪ between OSRM time and segment OSRM time.")
else:
    print("Fail to reject the null hypothesis: There is no significant
    ↪ difference between OSRM time and segment OSRM time.")
```

T-statistic: 30.29743310414474

P-value: 1.0892807362104113e-195

Reject the null hypothesis: There is a significant difference between OSRM time and segment OSRM time.

Find outliers in the numerical variables (you might find outliers in almost all the variables), and check it using visual analysis

```
[ ]: # List of numerical columns
numerical_cols = df.select_dtypes(include=['float64', 'int64']).columns

# Initialize a dictionary to store outlier information
```

```

outliers_dict = {}

for col in numerical_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Identify outliers
    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
    outliers_dict[col] = outliers

print(f"{col}: {len(outliers)} outliers found.")

```

```

start_scan_to_end_scan: 373 outliers found.
actual_distance_to_destination: 17992 outliers found.
actual_time: 16633 outliers found.
osrm_time: 17603 outliers found.
osrm_distance: 17816 outliers found.
segment_actual_time: 9298 outliers found.
segment_osrm_time: 6378 outliers found.
segment_osrm_distance: 4315 outliers found.
trip_creation_day: 0 outliers found.
trip_creation_month: 17518 outliers found.
trip_creation_year: 0 outliers found.
trip_creation_week: 0 outliers found.
trip_creation_hour: 0 outliers found.
time_taken: 373 outliers found.
od_total_time: 373 outliers found.

```

```

[ ]: plt.figure(figsize=(15, 10))

# Create box plots for each numerical variable
for i, col in enumerate(numerical_cols):
    plt.subplot(len(numerical_cols) // 3 + 1, 3, i + 1)
    sns.boxplot(y=df[col])
    plt.title(col)

plt.tight_layout()
plt.show()

```

/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:
FutureWarning:

SeriesGroupBy.grouper is deprecated and will be removed in a future version of pandas.

/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:
FutureWarning:

SeriesGroupBy.grouper is deprecated and will be removed in a future version of pandas.

/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:
FutureWarning:

SeriesGroupBy.grouper is deprecated and will be removed in a future version of pandas.

/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:
FutureWarning:

SeriesGroupBy.grouper is deprecated and will be removed in a future version of pandas.

/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:
FutureWarning:

SeriesGroupBy.grouper is deprecated and will be removed in a future version of pandas.

/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:
FutureWarning:

SeriesGroupBy.grouper is deprecated and will be removed in a future version of pandas.

/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:
FutureWarning:

SeriesGroupBy.grouper is deprecated and will be removed in a future version of pandas.

/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:
FutureWarning:

SeriesGroupBy.grouper is deprecated and will be removed in a future version of pandas.

/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:
FutureWarning:

SeriesGroupBy.grouper is deprecated and will be removed in a future version of pandas.

```
/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:  
FutureWarning:
```

SeriesGroupBy.grouper is deprecated and will be removed in a future version of pandas.

```
/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:  
FutureWarning:
```

SeriesGroupBy.grouper is deprecated and will be removed in a future version of pandas.

```
/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:  
FutureWarning:
```

SeriesGroupBy.grouper is deprecated and will be removed in a future version of pandas.

```
/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:  
FutureWarning:
```

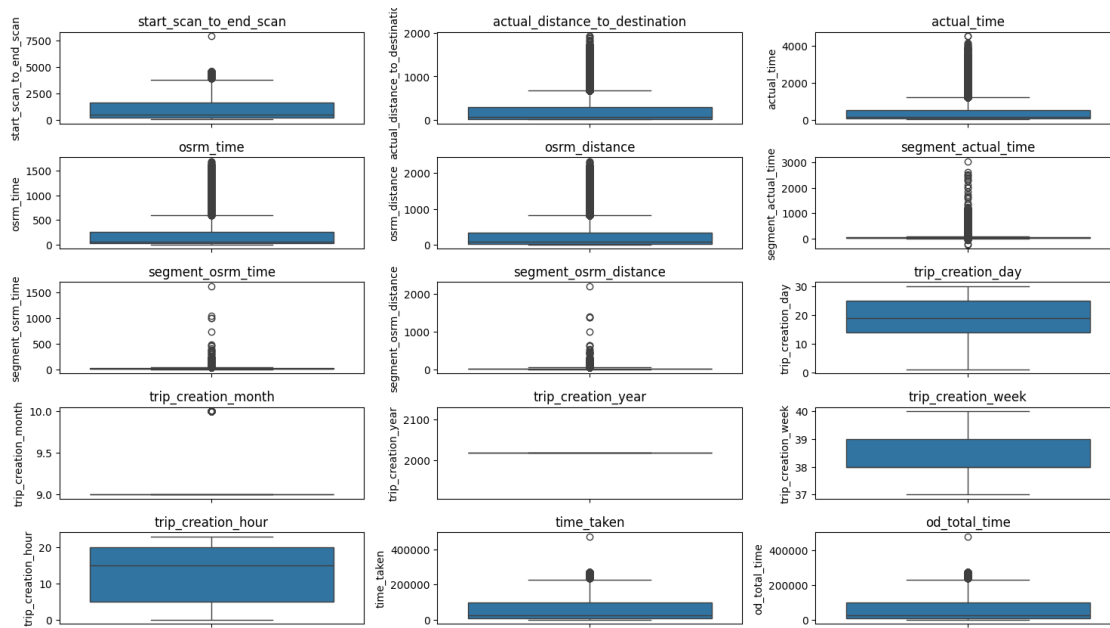
SeriesGroupBy.grouper is deprecated and will be removed in a future version of pandas.

```
/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:  
FutureWarning:
```

SeriesGroupBy.grouper is deprecated and will be removed in a future version of pandas.

```
/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:640:  
FutureWarning:
```

SeriesGroupBy.grouper is deprecated and will be removed in a future version of pandas.



Handle the outliers using the IQR method.

```
[ ]: numerical_cols = df.select_dtypes(include=['float64', 'int64']).columns

# Initialize a list to keep the indices of rows to drop
rows_to_drop = []

for col in numerical_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Identify outlier indices
    outlier_indices = df[(df[col] < lower_bound) | (df[col] > upper_bound)].
    ↪ index
    rows_to_drop.extend(outlier_indices)

# Drop duplicate indices
rows_to_drop = list(set(rows_to_drop))

# Drop the outliers from the DataFrame
df_cleaned = df.drop(index=rows_to_drop)

print(f"Original DataFrame shape: {df.shape}")
print(f"Cleaned DataFrame shape: {df_cleaned.shape}")
```

Original DataFrame shape: (144867, 33)
 Cleaned DataFrame shape: (100423, 33)

```
[ ]: # Cap Outliers (Alternative Method)
for col in numerical_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Cap the outliers
    df[col] = df[col].clip(lower=lower_bound, upper=upper_bound)
```

Do one-hot encoding of categorical variables (like route_type)

```
[ ]: # Identify categorical columns
categorical_cols = df.select_dtypes(include=['object']).columns
print("Categorical columns:", categorical_cols)
```

```
Categorical columns: Index(['route_schedule_uuid', 'trip_uuid', 'source_center',
'source_name',
    'destination_center', 'destination_name', 'destination_state',
    'destination_city', 'destination_place', 'source_state', 'source_city',
    'source_place'],
    dtype='object')
```

```
[ ]: # Perform one-hot encoding for the categorical variables
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

# Optionally, view the first few rows of the encoded DataFrame
print(df_encoded.head())
```

	data	trip_creation_time	route_type	od_start_time	\
0	training	2018-09-20 02:35:36.476840	Carting	2018-09-20 03:21:32.418600	
1	training	2018-09-20 02:35:36.476840	Carting	2018-09-20 03:21:32.418600	
2	training	2018-09-20 02:35:36.476840	Carting	2018-09-20 03:21:32.418600	
3	training	2018-09-20 02:35:36.476840	Carting	2018-09-20 03:21:32.418600	
4	training	2018-09-20 02:35:36.476840	Carting	2018-09-20 03:21:32.418600	

	od_end_time	start_scan_to_end_scan	\
0	2018-09-20 04:47:45.236797	86.0	
1	2018-09-20 04:47:45.236797	86.0	
2	2018-09-20 04:47:45.236797	86.0	
3	2018-09-20 04:47:45.236797	86.0	
4	2018-09-20 04:47:45.236797	86.0	

	actual_distance_to_destination	actual_time	osrm_time	osrm_distance	...	\
--	--------------------------------	-------------	-----------	---------------	-----	---

0	10.435660	14.0	11.0	11.9653 ...
1	18.936842	24.0	20.0	21.7243 ...
2	27.637279	40.0	28.0	32.5395 ...
3	36.118028	62.0	40.0	45.5620 ...
4	39.386040	68.0	44.0	54.2181 ...

	source_place_kalibari_D	source_place_kalmpuza_D	source_place_kankroli_D \
0	False	False	False
1	False	False	False
2	False	False	False
3	False	False	False
4	False	False	False

	source_place_keshod_DC	source_place_krshnPly_DC	source_place_lalaNGR_D \
0	False	False	False
1	False	False	False
2	False	False	False
3	False	False	False
4	False	False	False

	source_place_nagar_D	source_place_patna_D	source_place_unknown_place \
0	False	False	False
1	False	False	False
2	False	False	False
3	False	False	False
4	False	False	False

	source_place_ward9_D
0	False
1	False
2	False
3	False
4	False

[5 rows x 27131 columns]

```
[ ]: print(f"Original DataFrame shape: {df.shape}")
      print(f"Encoded DataFrame shape: {df_encoded.shape}")
```

Original DataFrame shape: (144867, 33)
 Encoded DataFrame shape: (144867, 27131)

```
[ ]: # Review the Resulting DataFrame

      print(f"Original DataFrame shape: {df.shape}")
      print(f"Encoded DataFrame shape: {df_encoded.shape}")
```

Original DataFrame shape: (144867, 33)

Encoded DataFrame shape: (144867, 27131)

Normalize/ Standardize the numerical features using MinMaxScaler or StandardScaler.

```
[ ]: # Initialize the MinMaxScaler
minmax_scaler = MinMaxScaler()

# List of numerical columns
numerical_cols = df.select_dtypes(include=['float64', 'int64']).columns

# Scale the numerical features
df[numerical_cols] = minmax_scaler.fit_transform(df[numerical_cols])

# Optional: View the first few rows of the scaled DataFrame
print(df.head())
```

```
      data      trip_creation_time \
0  training 2018-09-20 02:35:36.476840
1  training 2018-09-20 02:35:36.476840
2  training 2018-09-20 02:35:36.476840
3  training 2018-09-20 02:35:36.476840
4  training 2018-09-20 02:35:36.476840
```

```
      route_schedule_uuid route_type \
0  thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...  Carting
1  thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...  Carting
2  thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...  Carting
3  thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...  Carting
4  thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...  Carting
```

```
      trip_uuid source_center      source_name \
0  trip-153741093647649320  IND388121AAA  Anand_VUNagar_DC (Gujarat)
1  trip-153741093647649320  IND388121AAA  Anand_VUNagar_DC (Gujarat)
2  trip-153741093647649320  IND388121AAA  Anand_VUNagar_DC (Gujarat)
3  trip-153741093647649320  IND388121AAA  Anand_VUNagar_DC (Gujarat)
4  trip-153741093647649320  IND388121AAA  Anand_VUNagar_DC (Gujarat)
```

```
      destination_center      destination_name \
0  IND388620AAB  Khambhat_MotvdDPP_D (Gujarat)
1  IND388620AAB  Khambhat_MotvdDPP_D (Gujarat)
2  IND388620AAB  Khambhat_MotvdDPP_D (Gujarat)
3  IND388620AAB  Khambhat_MotvdDPP_D (Gujarat)
4  IND388620AAB  Khambhat_MotvdDPP_D (Gujarat)
```

```
      od_start_time ... source_city source_place \
0 2018-09-20 03:21:32.418600 ...  Anand  VUNagar_DC
1 2018-09-20 03:21:32.418600 ...  Anand  VUNagar_DC
2 2018-09-20 03:21:32.418600 ...  Anand  VUNagar_DC
```



```

3 2018-09-20 03:21:32.418600 ...      Anand      VUNagar_DC
4 2018-09-20 03:21:32.418600 ...      Anand      VUNagar_DC

```

```

      trip_creation_date  trip_creation_day  trip_creation_month \
0      2018-09-20      0.655172      0.0
1      2018-09-20      0.655172      0.0
2      2018-09-20      0.655172      0.0
3      2018-09-20      0.655172      0.0
4      2018-09-20      0.655172      0.0

```

```

      trip_creation_year  trip_creation_week  trip_creation_hour  time_taken \
0      0.0      0.333333      0.086957      0.01713
1      0.0      0.333333      0.086957      0.01713
2      0.0      0.333333      0.086957      0.01713
3      0.0      0.333333      0.086957      0.01713
4      0.0      0.333333      0.086957      0.01713

```

```

      od_total_time
0      0.017132
1      0.017132
2      0.017132
3      0.017132
4      0.017132

```

[5 rows x 33 columns]

```

[ ]: # Initialize the StandardScaler
      standard_scaler = StandardScaler()

      # Scale the numerical features
      df[numerical_cols] = standard_scaler.fit_transform(df[numerical_cols])

      # Optional: View the first few rows of the scaled DataFrame
      print(df.head())

```

```

      data      trip_creation_time \
0  training 2018-09-20 02:35:36.476840
1  training 2018-09-20 02:35:36.476840
2  training 2018-09-20 02:35:36.476840
3  training 2018-09-20 02:35:36.476840
4  training 2018-09-20 02:35:36.476840

```

```

      route_schedule_uuid  route_type \
0  thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...  Carting
1  thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...  Carting
2  thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...  Carting
3  thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...  Carting
4  thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...  Carting

```

	trip_uuid	source_center	source_name	\
0	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC	(Gujarat)
1	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC	(Gujarat)
2	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC	(Gujarat)
3	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC	(Gujarat)
4	trip-153741093647649320	IND388121AAA	Anand_VUNagar_DC	(Gujarat)

	destination_center	destination_name	\
0	IND388620AAB	Khambhat_MotvdDPP_D	(Gujarat)
1	IND388620AAB	Khambhat_MotvdDPP_D	(Gujarat)
2	IND388620AAB	Khambhat_MotvdDPP_D	(Gujarat)
3	IND388620AAB	Khambhat_MotvdDPP_D	(Gujarat)
4	IND388620AAB	Khambhat_MotvdDPP_D	(Gujarat)

	od_start_time	...	source_city	source_place	\
0	2018-09-20 03:21:32.418600	...	Anand	VUNagar_DC	
1	2018-09-20 03:21:32.418600	...	Anand	VUNagar_DC	
2	2018-09-20 03:21:32.418600	...	Anand	VUNagar_DC	
3	2018-09-20 03:21:32.418600	...	Anand	VUNagar_DC	
4	2018-09-20 03:21:32.418600	...	Anand	VUNagar_DC	

	trip_creation_date	trip_creation_day	trip_creation_month	\
0	2018-09-20	0.204383	0.0	
1	2018-09-20	0.204383	0.0	
2	2018-09-20	0.204383	0.0	
3	2018-09-20	0.204383	0.0	
4	2018-09-20	0.204383	0.0	

	trip_creation_year	trip_creation_week	trip_creation_hour	time_taken	\
0	0.0	-0.31023	-1.340026	-0.846071	
1	0.0	-0.31023	-1.340026	-0.846071	
2	0.0	-0.31023	-1.340026	-0.846071	
3	0.0	-0.31023	-1.340026	-0.846071	
4	0.0	-0.31023	-1.340026	-0.846071	

	od_total_time
0	-0.846067
1	-0.846067
2	-0.846067
3	-0.846067
4	-0.846067

[5 rows x 33 columns]

```
[ ]: # Review the scaled data
```

```
print(df.describe()) # This will show the mean and standard deviation for the
↳ scaled features
```

	trip_creation_time	od_start_time \
count	144867	144867
mean	2018-09-22 13:34:23.659819264	2018-09-22 18:02:45.855230720
min	2018-09-12 00:00:16.535741	2018-09-12 00:00:16.535741
25%	2018-09-17 03:20:51.775845888	2018-09-17 08:05:40.886155008
50%	2018-09-22 04:24:27.932764928	2018-09-22 08:53:00.116656128
75%	2018-09-27 17:57:56.350054912	2018-09-27 22:41:50.285857024
max	2018-10-03 23:59:42.701692	2018-10-06 04:27:23.392375
std	NaN	NaN

	od_end_time	start_scan_to_end_scan \
count	144867	1.448670e+05
mean	2018-09-23 10:04:31.395393024	-2.483787e-16
min	2018-09-12 00:50:10.814399	-9.096640e-01
25%	2018-09-18 01:48:06.410121984	-7.732309e-01
50%	2018-09-23 03:13:03.520212992	-4.945590e-01
75%	2018-09-28 12:49:06.054018048	6.520599e-01
max	2018-10-08 03:00:24.353479	2.789996e+00
std	NaN	1.000003e+00

	actual_distance_to_destination	actual_time	osrm_time \
count	1.448670e+05	1.448670e+05	1.448670e+05
mean	-2.432778e-17	-5.336415e-17	-2.393539e-17
min	-7.744347e-01	-8.236212e-01	-8.175146e-01
25%	-7.134198e-01	-7.205011e-01	-7.155733e-01
50%	-5.316364e-01	-5.216264e-01	-5.359626e-01
75%	4.058794e-01	4.138210e-01	4.009260e-01
max	2.084828e+00	2.115304e+00	2.075675e+00
std	1.000003e+00	1.000003e+00	1.000003e+00

	osrm_distance	segment_actual_time	segment_osrm_time \
count	1.448670e+05	1.448670e+05	1.448670e+05
mean	2.130642e-16	5.181424e-16	-5.402630e-17
min	-7.889023e-01	-2.384257e+00	-1.953781e+00
25%	-7.143980e-01	-6.575279e-01	-7.268756e-01
50%	-5.411633e-01	-1.395091e-01	-5.765437e-02
75%	4.020286e-01	4.936249e-01	5.000300e-01
max	2.076669e+00	2.220354e+00	2.340388e+00
std	1.000003e+00	1.000003e+00	1.000003e+00

	segment_osrm_distance	trip_creation_date \
count	1.448670e+05	144867
mean	5.307968e-16	2018-09-22 00:25:08.915073792
min	-1.943570e+00	2018-09-12 00:00:00

25%	-8.773453e-01	2018-09-17 00:00:00
50%	1.334746e-01	2018-09-22 00:00:00
75%	5.133415e-01	2018-09-27 00:00:00
max	2.599372e+00	2018-10-03 00:00:00
std	1.000003e+00	NaN

	trip_creation_day	trip_creation_month	trip_creation_year	\
count	1.448670e+05	144867.0	144867.0	
mean	1.302713e-16	0.0	0.0	
min	-2.207172e+00	0.0	0.0	
25%	-5.571608e-01	0.0	0.0	
50%	7.745874e-02	0.0	0.0	
75%	8.390022e-01	0.0	0.0	
max	1.473622e+00	0.0	0.0	
std	1.000003e+00	0.0	0.0	

	trip_creation_week	trip_creation_hour	time_taken	od_total_time
count	1.448670e+05	1.448670e+05	1.448670e+05	1.448670e+05
mean	1.255627e-16	-3.609928e-17	6.121182e-17	8.318530e-17
min	-1.344641e+00	-1.591614e+00	-9.094598e-01	-9.094631e-01
25%	-3.102299e-01	-9.626429e-01	-7.732227e-01	-7.732228e-01
50%	-3.102299e-01	2.953002e-01	-4.948090e-01	-4.948049e-01
75%	7.241816e-01	9.242717e-01	6.524915e-01	6.524844e-01
max	1.758593e+00	1.301655e+00	2.791063e+00	2.791045e+00
std	1.000003e+00	1.000003e+00	1.000003e+00	1.000003e+00